AC DC OPF

ACDC-OpFlow Manual V0.1.4

Haixiao Li Azadeh Kermansaravi Aleksandra Lekić

October 26, 2026

Contents

1 Overview **2**
1.1 Motivation . . . 2
1.2 Configuration . . . 2
1.2.1 Dependencies . . . 2
1.2.2 Installation steps . . . 3
1.2.3 Code structure . . . 4

2 OPF Formulation **5**
2.1 Nomenclature . . . 5
2.2 Original nonlinear formulation . . . 7
2.3 Convex relaxed formulation . . . 10
2.4 Voltage phase recovery . . . 13

3 Code Implementation **13**
3.1 Data preparation . . . 13
3.2 Parameter processing . . . 19
3.3 Variable definition . . . 19
3.4 Constraint construction . . . 22

4 Examples **25**
4.1 Running a case . . . 25
4.2 Benchmark . . . 26
4.2.1 Performance comparison . . . 26
4.2.2 Discussion . . . 27

A Additional Discussion **30**
A.1 Power flow accuracy . . . 30
A.2 Integrate into Harmony . . . 32

1 Overview

1.1 Motivation

AC/VSC-MTDC (Voltage Source Converter-based Multi-terminal DC) grids play a crucial role in modern power systems by enhancing grid stability, flexibility, and grid-friendly interconnection. VSC-MTDC allows independent active and reactive power control, enabling seamless connection of AC grids in different regions. The scheduling of a power system consisting of multiple interconnected AC grids and a VSC-MTDC grid is significant [1], [2], [3]. As depicted in Fig. 1.1, we need a technique to help enhance the rationality and optimality of scheduling, which involves solving the problems known as the AC/DC optimal power flow (OPF).

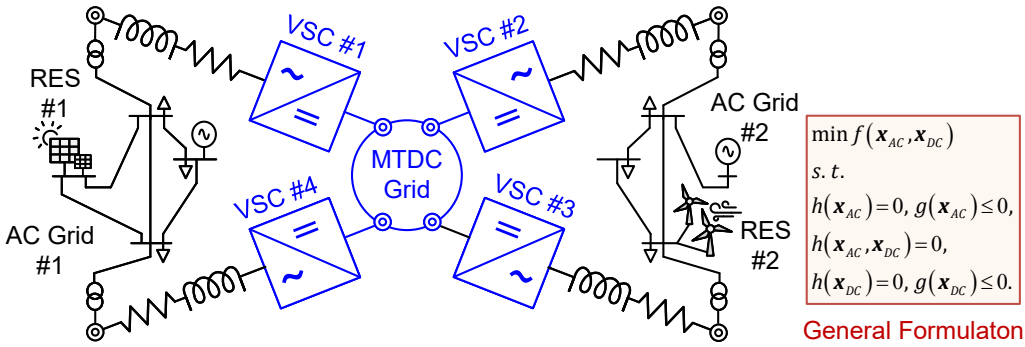


Figure 1.1: Overview of the AC/DC OPF problem

This section provides a concise guide on environment configuration, core code structure, running the code, and accessing results. It serves as a basic introduction to help users to get started with **ACDC-OpFlow**.

1.2 Configuration

1.2.1 Dependencies

ACDC-OpFlow is developed using languages: Matlab, Julia, Python, Cpp. For each programming environment, we provide recommendations on the dependency versions used in our implementation. While users may install later versions of these dependencies, it is advised to verify compatibility.

- Matlab r2021b: Yalmip 20230622.
- Julia v1.10: CSV 0.10.15 | DataFrames 1.7.0 | JuMP 1.23.3 | Gurobi 1.0.0 | Graphs 1.12.1 | GraphPlot 0.6.1 | GLMakie 0.11.4 | ColorSchemes 3.29.0 | Colors 0.12.11.
- Python v3.12 : numpy 2.2.1 | pandas 2.2.3 | scipy 1.15.1 | pyomo 6.8.2 | networkx 3.4.2 | matplotlib 3.10.0 | gurobipy 11.0.1.
- Cpp : Gurobi C++ API | Eigen3 3.4.0 | matplotplusplus 3.4.0.

- Gurobi GUROBI OPTIMIZATION : Gurobi 9.5.2.
- Matpower MATPOWER : Matpower 7.1.

1.2.2 Installation steps

To install required toolboxes in Matlab environment, users can download the toolbox directly from the official website. Then, users can add the toolbox folders to Matlab path, using the following commands:

```
>> addpath(genpath('path/to/YALMIP'))
>> ...
>> savepath
```

To install required packages in Julia environment, users can use the following commands:

```
>> using Pkg
>> Pkg.add("JuMP")
>> Pkg.add("CSV")
>> ...
```

Alternatively, users may use the package mode (triggered by pressing the `]` key in the Julia REPL):

```
>> (@v1.10) pkg> add JuMP
>> (@v1.10) pkg> add CSV
>> ...
```

To install required packages in Python environment, users can use the following commands if with Anaconda[1], installing packages into isolated environments:

```
>> conda create -n acdcOpflow-env python=3.12
>> conda activate acdcOpflow-env
>> conda install numpy pandas matplotlib ...
```

Also, users can install packages either globally or within virtual environments:

```
>> pip install numpy pandas matplotlib ...
```

To install required libraries in Cpp, we recommend using vcpkg[2] to simplifies the process of acquiring and managing Cpp dependencies. The commands are:

```
>> ./vcpkg install eigen3 boost
>> ./vcpkg install matplotplusplus
>> ...
```

[1] https://www.anaconda.com/
[2] https://vcpkg.io/en/

1.2.3 Code structure

As presented in Fig. 1.2, under the framework of **ACDC-OpFlow**, every coding language follows a unified workflow: First, we prepare input data related to AC and DC grids, which are all saved in structured comma-separated value (CSV) files. Secondly, extracting key parameter information from .csv data files and handling them to form the specific parameter vectors (matrices). Third, based on predefined parameter vectors (matrices), the AC/DC OPF model is constructed and subsequently solved using the Gurobi solver. Lastly, the optimal power flow results are organized and output in text format, accompanied by a visual presentation to facilitate the observation of the results. For each coding language, the included main functions are listed as Fig. 1.3.

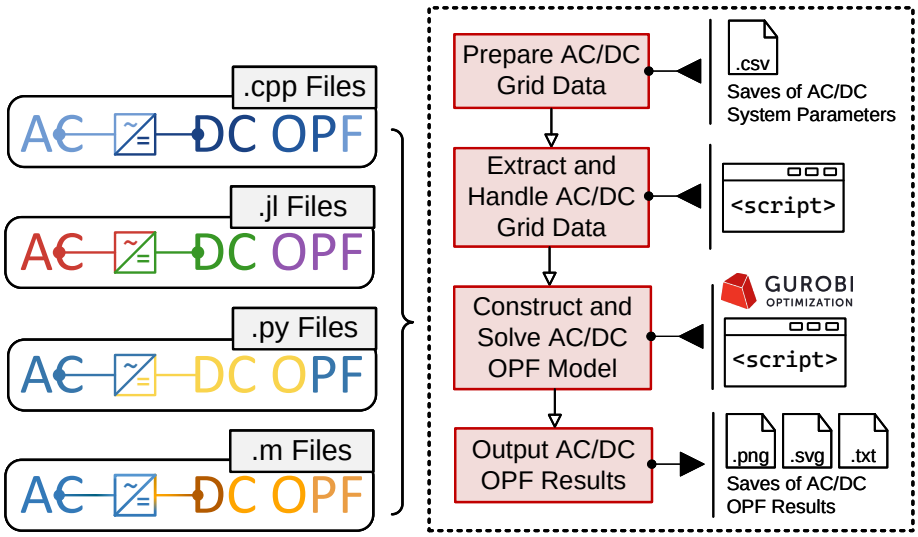


Figure 1.2: Unified workflow in **ACDC-OpFlow**

Core functions

- `create_ac`: AC system data loading
- `create_dc`: DC system data loading
- `params_ac`: AC system parameter extraction
- `params_dc`: DC system parameter extraction
- `solve_opf`: AC/DC OPF implementation
- `viz_opf`: AC/DC OPF result visulization

Figure 1.3: Main functions in **ACDC-OpFlow**

2 OPF Formulation

2.1 Nomenclature

This section introduces the mathematical formulation of the AC/DC OPF used in **ACDC-OpFlow**. The fundamental constraint and objective functions to describe the optimal steady-state behavior for the AC/DC power system are introduced. Symbols appearing in subsequent formulas can be found below.

Indexes, Sets, super/subscript

$(\bar{\cdot})/(\underline{\cdot})$ -The upper/lower bound of the corresponding optimization variables and parameters

$|\cdot|$ -Magnitude

$\mathcal{L}$ -Branch set

$\mathcal{N}$ -Node set

AC -Alternating current

$cost$ -Generation cost

$droop$ -Droop parameters

gen -Generator

$h,\ i,\ j$ -System node

ij -System branch

$load$ -Load

$loss$ -Power loss

$mppt$ -Maximum Power Point Tracking

$MTDC$ -Multi-terminal direct current

ref -Reference value

res -Renewable energy source

VSC -Voltage source converter

Variables and Parameters Related to the AC Grid

θ_{ij}^{AC} -Phase difference for AC branch ij

$c_{i,2}^{AC}/c_{i,1}^{AC}/c_{i,0}^{AC}$ -Quadratic/Linear/Constant generator generation coefficient at AC bus i

c_{ij}^{AC}/s_{ij}^{AC} -Optimization variables linked with the AC nodal voltage

$e_{i,2}^{AC}/e_{i,1}^{AC}/e_{i,0}^{AC}$ -Quadratic/Linear/Constant REs generation coefficient at AC bus i

G_{ij}^{AC}/B_{ij}^{AC} -Real/Imaginary part of AC bus admittance matrix

g_{ij}^{AC}/b_{ij}^{AC} -Conductance/Susceptance of AC branch ij

p_{ij}^{AC}/q_{ij}^{AC} -Real/Reactive power for AC branch ij

$p_{i,A2V}^{AC}/q_{i,A2V}^{AC}$ -Real/Reactive power delivered from the AC grid to the VSC station at AC bus i

$p_{i,gen}^{AC}/q_{i,gen}^{AC}$ -Real/Reactive power outputs of generator at AC bus i

$p_{i,load}^{AC}/q_{i,load}^{AC}$ -Real/Reactive power loads at AC bus i

$p_{i,res}^{AC}/q_{i,res}^{AC}$ -Real/Reactive power of RESs at AC bus i

p_{i}^{AC}/q_{i}^{AC} -Real/Reactive power injection at AC bus i

$s_{i,res}^{AC}$ -Apparent power of RES at AC bus i

v_{i}^{AC} -AC bus i voltage

Variables and Parameters Related to the MTDC Grid

$k_{i,droop}^{MTDC}$ -Droop slope at MTDC bus i

l_{jh}^{MTDC} -Squared current of MTDC branch jh

$p_{i,ref}^{MTDC}$ -Power reference of DC power control/DC droop control at MTDC bus i

p_{jh}^{MTDC} -Power of MTDC branch jh

u_{j}^{MTDC} -Squared MTDC bus j voltage

v_{j}^{MTDC} -MTDC bus j voltage

$V_{i,ref}^{MTDC}$ -Voltage reference of DC voltage control/DC droop control at MTDC bus i

Variables and Parameters Related to the VSC Station

$\theta_{s}^{VSC}/\theta_{f}^{VSC}/\theta_{c}^{VSC}$ -VSC bus $s/f/c$ phase

$a_{loss}^{VSC}/b_{loss}^{VSC}/c_{loss}^{VSC}$ -Constant/Linear/Quadratic coefficient of VSC cpower loss function

b_{f}^{VSC} -Susceptance for VSC bus f to ground

$b_{sf}^{VSC}/b_{fc}^{VSC}$ -Susceptance for VSC branch sf/fc

$c_{sf}^{VSC}/s_{sf}^{VSC}/c_{cf}^{VSC}/s_{cf}^{VSC}$ -Optimization variables linked with the VSC nodal voltage

$g_{sf}^{VSC}/g_{fc}^{VSC}$ -Conductance for VSC branch sf/fc

I_{c}^{VSC} -Phase current at VSC bus c

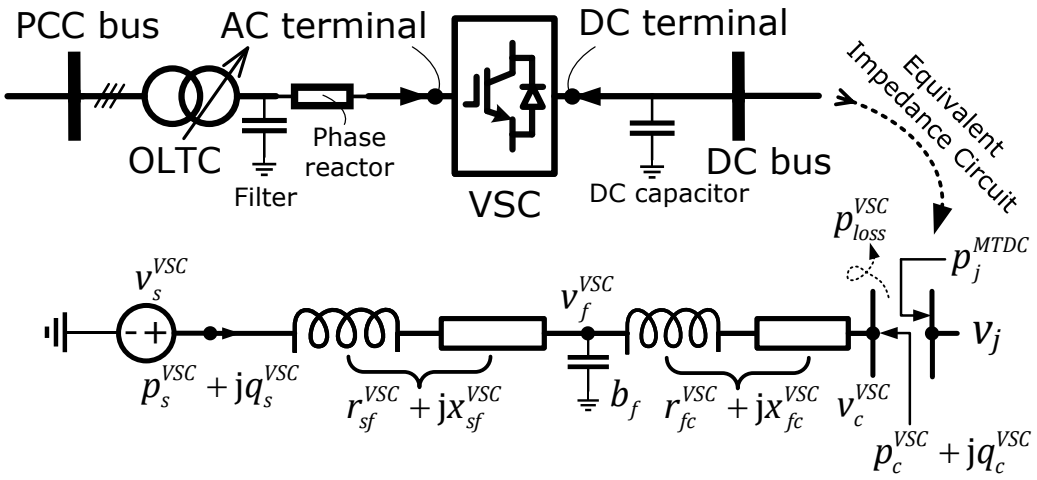


Figure 2.1: Equivalent impedance model of VSC

l_c^{VSC} -Squared phase current at VSC bus c

p_{loss}^{VSC} -VSC power loss

$q_{s,ref}^{VSC}$ -Power reference of reactive power control at VSC bus s

$v_s^{VSC}/v_f^{VSC}/v_c^{VSC}$ -VSC bus $s/f/c$ voltage

$v_{s,ref}^{VSC}$ -Voltage reference of AC voltage control at VSC bus s

2.2 Original nonlinear formulation

The original AC/DC OPF formulation contains is formulated as a nonlinear programming (NLP) problem due to the presence of several nonlinear constraints, such as nonlinear power flow constraints and quadratic converter loss constraints. Fig. 2.1 is provided to help understanding power flow formulation at the AC/DC coupling points. The explicit formulation of the original AC/DC OPF model is prsented as below.

Original AC/MTDC OPF Formulation [NLP]

Variables:

$p_i^{AC}, q_i^{AC}, p_{i,gen}^{AC}, q_{i,gen}^{AC}, p_{i,res}^{AC}, q_{i,res}^{AC}, s_{i,res}^{AC}, p_{i,A2V}^{AC}, q_{i,A2V}^{AC}, \theta_i^{AC}, v_i^{AC}$	$i \in \mathcal{N}^{AC}$
p_{ij}^{AC}, q_{ij}^{AC}	$(i,j) \in \mathcal{L}^{AC}$
p_j^{MTDC}, v_j^{MTDC}	$j \in \mathcal{N}^{MTDC}$
p_{jh}^{MTDC}	$(j,h) \in \mathcal{L}^{MTDC}$
$p_s^{VSC}, q_s^{VSC}, \theta_s^{VSC}, v_s^{VSC}$	$s \in \mathcal{N}^{VSC}$
$p_c^{VSC}, q_c^{VSC}, \theta_c^{VSC}, v_c^{VSC}, I_c^{VSC}$	$c \in \mathcal{N}^{VSC}$
$q_f^{VSC}, \theta_f^{VSC}, v_f^{VSC}$	$f \in \mathcal{N}^{VSC}$

AC Constraints:

$$p_{ij}^{AC} = g_{ij}^{AC} \left|v_i^{AC}\right|^2 - \left|v_i^{AC}\right| \left|v_j^{AC}\right| \left(g_{ij}^{AC} \cos\theta_{ij}^{AC} + b_{ij}^{AC} \sin\theta_{ij}^{AC}\right), \quad i,j \in \mathcal{N}^{AC}, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.1a}$$

$$q_{ij}^{AC} = -b_{ij}^{AC} \left|v_i^{AC}\right|^2 + \left|v_i^{AC}\right| \left|v_j^{AC}\right| \left(b_{ij}^{AC} \cos\theta_{ij}^{AC} - g_{ij}^{AC} \sin\theta_{ij}^{AC}\right), \quad i,j \in \mathcal{N}^{AC}, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.1b}$$

$$p_i^{AC} = \sum_{(i,j)} p_{ij}^{AC} + \left(\sum_j G_{ij}^{AC}\right) (v_i^{AC})^2, \quad i,j \in \mathcal{N}^{AC}, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.1c}$$

$$q_i^{AC} = \sum_{(i,j)} q_{ij}^{AC} + \left(\sum_j -B_{ij}^{AC}\right) (v_i^{AC})^2, \quad i,j \in \mathcal{N}^{AC}, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.1d}$$

$$p_i^{AC} = p_{i,gen}^{AC} - p_{i,load}^{AC} - p_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.2a}$$

$$q_i^{AC} = q_{i,gen}^{AC} - q_{i,load}^{AC} - q_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.2b}$$

$$\underline{p}_{i,gen}^{AC} \le p_{i,gen}^{AC} \le \overline{p}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.3a}$$

$$\underline{q}_{i,gen}^{AC} \le q_{i,gen}^{AC} \le \overline{q}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.3b}$$

$$\underline{\left|v_i^{AC}\right|} \le \left|v_i^{AC}\right| \le \overline{\left|v_i^{AC}\right|}, \quad i \in \mathcal{N}^{AC} \tag{2.4}$$

$$0 \le p_{i,res}^{AC} \le \bar{p}_{i,res}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.5a}$$

$$\left(p_{i,res}^{AC}\right)^2 + \left(q_{i,res}^{AC}\right)^2 \le \left(\bar{s}_{i,res}^{AC}\right)^2 \tag{2.5b}$$

MTDC Constraints:

$$p_{jh}^{MTDC} = \rho^{MTDC} \cdot v_j^{MTDC} \left(v_j^{MTDC} - v_h^{MTDC}\right) y_{jh}^{MTDC}, \quad j,h \in \mathcal{N}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.6a}$$

$$p_j^{MTDC} = \sum_{(j,h)} p_{jh}^{MTDC}, \quad j,h \in \mathcal{N}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.6b}$$

$$\underline{v}_j^{MTDC} \le v_j^{MTDC} \le \overline{v}_j^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \tag{2.7}$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \tag{2.8a}$$

$$v_j^{MTDC} = v_{j,ref}^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \tag{2.8b}$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC} - \frac{1}{k_{j,droop}^{MTDC}} \left(v_j^{MTDC} - v_{j,ref}^{MTDC}\right), \quad j \in \mathcal{N}^{MTDC} \tag{2.8c}$$

VSC Constraints:

$$p_s^{VSC} = p_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC}, \quad i \in \mathcal{N}^{AC} \tag{2.9a}$$

$$q_s^{VSC} = q_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC}, \quad i \in \mathcal{N}^{AC} \tag{2.9b}$$

$$\begin{aligned} p_s^{VSC} = \left|v_s^{VSC}\right|^2 g_{sf}^{VSC} - \left|v_s^{VSC}\right| \left|v_f^{VSC}\right| \left[g_{sf}^{VSC} \cos\left(\theta_s^{VSC} - \theta_f^{VSC}\right)\right. \\ \left. + b_{sf}^{VSC} \sin\left(\theta_s^{VSC} - \theta_f^{VSC}\right)\right], \quad s, f \in \mathcal{N}^{VSC} \end{aligned} \tag{2.10a}$$

$$\begin{aligned} q_s^{VSC} = -\left|v_s^{VSC}\right|^2 b_{sf}^{VSC} - \left|v_s^{VSC}\right| \left|v_f^{VSC}\right| \left[g_{sf}^{VSC} \cos\left(\theta_s^{VSC} - \theta_f^{VSC}\right)\right. \\ \left. - b_{sf}^{VSC} \sin\left(\theta_s^{VSC} - \theta_f^{VSC}\right)\right], \quad s, f \in \mathcal{N}^{VSC} \end{aligned} \tag{2.10b}$$

$$\begin{aligned} p_c^{VSC} = \left|v_c^{VSC}\right|^2 g_{fc}^{VSC} + \left|v_f^{VSC}\right| \left|v_c^{VSC}\right| \left[g_{fc}^{VSC} \cos\left(\theta_f^{VSC} - \theta_c^{VSC}\right)\right. \\ \left. - b_{fc}^{VSC} \sin\left(\theta_f^{VSC} - \theta_c^{VSC}\right)\right], \quad f, c \in \mathcal{N}^{VSC} \end{aligned} \tag{2.11a}$$

$$\begin{aligned} q_c^{VSC} = -\left|v_c^{VSC}\right|^2 b_{fc}^{VSC} + \left|v_f^{VSC}\right| \left|v_c^{VSC}\right| \left[g_{fc}^{VSC} \sin\left(\theta_f^{VSC} - \theta_c^{VSC}\right)\right. \\ \left. + b_{fc}^{VSC} \cos\left(\theta_f^{VSC} - \theta_c^{VSC}\right)\right], \quad f, c \in \mathcal{N}^{VSC} \end{aligned} \tag{2.11b}$$

$$q_f^{VSC} = -\left|v_f^{VSC}\right|^2 b_f^{VSC}, \quad f \in \mathcal{N}^{VSC} \tag{2.12}$$

$$p_c^{VSC} + p_{loss}^{VSC} + p_j^{MTDC} = 0, \quad j \in \mathcal{N}^{MTDC}, \quad c \in \mathcal{N}^{VSC} \tag{2.13}$$

$$p_{loss}^{VSC} = a_{loss}^{VSC} + b_{loss}^{\mathrm{VSC}} \left|I_c^{VSC}\right| + c_{loss}^{VSC} \left|I_c^{VSC}\right|^2, \quad c \in \mathcal{N}^{VSC} \tag{2.14a}$$

$$0 \le \left|I_c^{VSC}\right| \le \left|\overline{I_c^{VSC}}\right|, \quad c \in \mathcal{N}^{VSC} \tag{2.14b}$$

$$(v_c^{VSC})^2 \cdot \left|I_c^{VSC}\right|^2 = (p_c^{VSC})^2 + (q_c^{VSC})^2 \quad c \in \mathcal{N}^{VSC} \tag{2.15}$$

$$v_s^{VSC} = v_{s,ref}^{VSC}, \quad s \in \mathcal{N}^{VSC} \tag{2.16a}$$

$$q_s^{VSC} = q_{s,ref}^{VSC}, \quad s \in \mathcal{N}^{VSC} \tag{2.16b}$$

Goal:

$$\min \sum_i \left\{ c_{i,2}^{AC} \left(p_{i,gen}^{AC}\right)^2 + c_{i,1}^{AC} p_{i,gen}^{AC} + c_{i,0}^{AC} + e_{i,2}^{AC} \left(p_{i,res}^{AC}\right)^2 + e_{i,1}^{AC} p_{i,res}^{AC} + e_{i,0}^{AC} \right\}, \quad i \in \mathcal{N}^{AC} \tag{2.17}$$

- **AC Constraints**: ✓(2.1a)-(2.1d) are power flow constraints. ✓(2.2a)-(2.2b) are nodal power balance equations. ✓(2.3a)-(2.3b) are generator power output limits. ✓(2.4) is the voltage magnitude limit. ✓(2.4) is the voltage magnitude limit. ✓(2.5a)-(2.5b) regulates the power output constraint of RESs.

- **MTDC Constraints**: ✓(2.6a)-(2.6b) are power flow constraints. ✓(2.7) is the voltage magnitude limit. ✓(2.8a)-(2.8c) represents the VSC control mode at DC terminal. Note that only one of them is hold depending on the chosen control model.

- **VSC Constraint**: ✓(2.9a)-(2.9b) are the active and reactive power couplings between the AC grid and VSC. ✓(2.10a)-(2.10b) are the active and reactive power injections from the view of node s.✓(2.11a)-(2.11b) are the active and reactive power injections from the view of node c. ✓(2.12) is the reactive power injection from the view of node f. If we consider the latest converters with the MMC topology, the filter is not required to install at node f and $b_f^{vsc} = 0$.✓(2.13) is the active power coupling between the MTDC grid and VSC.✓(2.14a)-(2.14b) are related to the converter loss function. ✓(2.15) denote the voltage and current relationship from the view of node c. ✓(2.16a)-(2.16b) represent the VSC control mode at AC PCC. Note that only one of them holds depending on the chosen control model.

- **Goal**: ✓(2.17) represent minimizing power generation costs.

2.3 Convex relaxed formulation

The original AC/DC OPF problem is NLP. Such a problem is not favored due to their limited scalability and computational challenges. Give these reasons, we formulate a convex relaxed version of the original AC/DC OPF problem. Our demo code is based on the convex relaxed version as below.

Convex Relaxed AC/MTDC OPF Formulation [SOCP]

Variables:

$p_i^{AC}, q_i^{AC}, p_{i,gen}^{AC}, q_{i,gen}^{AC}, p_{i,A2V}^{AC}, q_{i,A2V}^{AC}, p_{i,res}^{AC}, q_{i,res}^{AC}, s_{i,res}^{AC}$	$i \in \mathcal{N}^{AC}$	
$p_{ij}^{AC}, q_{ij}^{AC}, s_{ij}^{AC}, c_{ij}^{AC}$	$(i,j) \in \mathcal{L}^{AC}$	
p_j^{MTDC}, v_j^{MTDC}	$j \in \mathcal{N}^{MTDC}$	
$p_{jh}^{MTDC}, l_{jh}^{MTDC}$	$(j,h) \in \mathcal{L}^{MTDC}$	
p_s^{VSC}, q_s^{VSC}	$s \in \mathcal{N}^{MTDC}$	
$p_c^{VSC}, q_c^{VSC}, I_c^{VSC}, l_c^{VSC}$	$c \in \mathcal{N}^{MTDC}$	
q_f^{VSC}	$f \in \mathcal{N}^{MTDC}$	
$c_{mn}^{VSC}, c_{mn}^{VSC}$	$m,n \in \{s,f,c\}$	$s,f,c \in \mathcal{N}^{MTDC}$

AC Constraints:

$$p_{ij}^{AC} = G_{ij}\left(c_{ii}^{AC} - c_{ij}^{AC}\right) + B_{ij}s_{ij}^{AC}, \quad i,j \in \mathcal{N}^{AC}, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.17a}$$

$$q_{ij}^{AC} = -B_{ij}\left(c_{ii}^{AC} - c_{ij}^{AC}\right) + G_{ij}s_{ij}^{AC}, \quad i,j \in \mathcal{N}^{AC}, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.17b}$$

$$p_i^{AC} = c_{ii}^{AC}G_{ii} + \sum_j \left(c_{ij}^{AC}G_{ij} - s_{ij}^{AC}B_{ij}\right), \quad i,j \in \mathcal{N}^{AC} \tag{2.17c}$$

$$q_i^{AC} = -c_{ii}^{AC}B_{ii} - \sum_j \left(c_{ij}^{AC}B_{ij} + s_{ij}^{AC}G_{ij}\right), \quad i,j \in \mathcal{N}^{AC} \tag{2.17d}$$

$$c_{ij}^{AC} = c_{ji}^{AC}, \quad i,j \in \mathcal{N}^{AC} \tag{2.17e}$$

$$s_{ij}^{AC} = -s_{ji}^{AC}, \quad i,j \in \mathcal{N}^{AC} \tag{2.17f}$$

$$(c_{ij}^{AC})^2 + (s_{ij}^{AC})^2 \leq c_{ii}^{AC}c_{jj}^{AC}, \quad i,j \in \mathcal{N}^{AC} \tag{2.17g}$$

$$p_i^{AC} = p_{i,gen}^{AC} - p_{i,load}^{AC} - p_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.18a}$$

$$q_i^{AC} = q_{i,gen}^{AC} - q_{i,load}^{AC} - q_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.18b}$$

$$\underline{p}_{i,gen}^{AC} \leq p_{i,gen}^{AC} \leq \overline{p}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.19a}$$

$$\underline{q}_{i,gen}^{AC} \leq q_{i,gen}^{AC} \leq \overline{q}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.19b}$$

$$\underline{\left|v_i^{AC}\right|}^2 \leq c_{ii}^{AC} \leq \overline{\left|v_i^{AC}\right|}^2, \quad i \in \mathcal{N}^{AC} \tag{2.20}$$

$$0 \leq p_{i,res}^{AC} \leq \bar{p}_{i,res}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.21a}$$

$$-\bar{s}_{i,res}^{AC} \leq \cos\left(\frac{k}{N}\pi\right)p_{i,res}^{AC} + \sin\left(\frac{k}{N}\pi\right)q_{i,res}^{AC} \leq \bar{s}_{i,res}^{AC}, \quad \forall k \in \{1,2,\cdots N\}, \quad i \in \mathcal{N}^{AC} \tag{2.21b}$$

MTDC Constraints:

$$p_j^{MTDC} = \sum_{(j,h)} p_{jh}^{MTDC}, \quad j \in \mathcal{N}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.22a}$$

$$p_{jh}^{MTDC} + p_{hj}^{MTDC} = r_{jh}^{MTDC}l_{jh}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.22b}$$

$$u_j^{MTDC} - u_h^{MTDC} = r_{jh}^{MTDC}\left(p_{jh}^{MTDC} - p_{hj}^{MTDC}\right), \quad j,h \in \mathcal{N}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.22c}$$

$$p_{jh}^{MTDC} \leq l_{jh}^{MTDC}u_j^{MTDC} \quad j \in \mathcal{N}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.22d}$$

$$\underline{u}_j^{MTDC} \leq u_j^{MTDC} \leq \overline{u}_j^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \tag{2.23}$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \tag{2.24a}$$

$$u_j^{MTDC} = \left(v_{j,ref}^{MTDC}\right)^2, \quad j \in \mathcal{N}^{MTDC} \tag{2.24b}$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC} - \frac{1}{k_{j,droop}^{MTDC}}\left(\frac{1}{2} + \frac{1}{2}u_j^{MTDC} - v_{j,ref}^{MTDC}\right), \quad j \in \mathcal{N}^{MTDC} \tag{2.24c}$$

VSC constraints:

$$p_s^{VSC} = p_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC} \tag{2.25a}$$
$$q_s^{VSC} = q_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC} \tag{2.25b}$$
$$p_s^{VSC} = c_{ss}^{VSC} g_{sf}^{VSC} - c_{sf}^{VSC} g_{sf}^{VSC} + s_{sf}^{VSC} b_{sf}^{VSC}, \quad s, f \in \mathcal{N}^{MTDC} \tag{2.26a}$$
$$q_s^{VSC} = -c_{ss}^{VSC} b_{sf}^{VSC} + c_{sf}^{VSC} b_{sf}^{VSC} + s_{sf}^{VSC} g_{sf}^{VSC}, \quad s, f \in \mathcal{N}^{MTDC} \tag{2.26b}$$
$$c_{sf}^{VSC} = c_{fs}^{VSC}, \quad s, f \in \mathcal{N}^{AC} \tag{2.26c}$$
$$s_{sf}^{VSC} = -s_{fs}^{VSC}, \quad s, f \in \mathcal{N}^{VSC} \tag{2.26d}$$
$$(c_{sf}^{VSC})^2 + (s_{sf}^{VSC})^2 \leq c_{ss}^{VSC} c_{ff}^{VSC}, \quad s, f \in \mathcal{N}^{AC} \tag{2.26e}$$
$$p_c^{VSC} = c_{cc}^{VSC} g_{cf}^{VSC} - c_{cf}^{VSC} g_{cf}^{VSC} + s_{cf}^{VSC} b_{cf}^{VSC}, \quad c, f \in \mathcal{N}^{MTDC} \tag{2.27a}$$
$$q_c^{VSC} = -c_{cc}^{VSC} b_{cf}^{VSC} + c_{cf}^{VSC} b_{cf}^{VSC} + s_{cf}^{VSC} g_{cf}^{VSC}, \quad c, f \in \mathcal{N}^{MTDC} \tag{2.27b}$$
$$c_{cf}^{VSC} = c_{fc}^{VSC}, \quad c, f \in \mathcal{N}^{VSC} \tag{2.27c}$$
$$s_{cf}^{VSC} = -s_{fc}^{VSC}, \quad c, f \in \mathcal{N}^{VSC} \tag{2.27d}$$
$$(c_{cf}^{VSC})^2 + (s_{cf}^{VSC})^2 \leq c_{cc}^{VSC} c_{ff}^{VSC}, \quad c, f \in \mathcal{N}^{VSC} \tag{2.27e}$$
$$q_f^{VSC} = -c_{ff}^{VSC} b_f^{VSC}, \quad f \in \mathcal{N}^{VSC} \tag{2.28}$$
$$p_c^{VSC} + p_{loss}^{VSC} + p_j^{MTDC} = 0, \quad j \in \mathcal{N}^{MTDC}, \quad c \in \mathcal{N}^{VSC} \tag{2.29}$$
$$p_{loss}^{VSC} = a_{loss}^{VSC} + b_{loss}^{\mathrm{V\,S\,C}} \left| I_c^{VSC} \right| + c_{loss}^{VSC} l_c^{VSC}, \quad c \in \mathcal{N}^{VSC} \tag{2.30a}$$
$$0 \leq \left| I_c^{VSC} \right| \leq \left| \overline{I_c^{VSC}} \right|, \quad c \in \mathcal{N}^{VSC} \tag{2.30b}$$
$$0 \leq l_c^{VSC} \leq \left| \overline{I_c^{VSC}} \right|^2, \quad c \in \mathcal{N}^{VSC} \tag{2.30c}$$
$$\left| I_c^{VSC} \right|^2 \leq l_c^{VSC}, \quad c \in \mathcal{N}^{VSC} \tag{2.31a}$$
$$(p_c^{VSC})^2 + (q_c^{VSC})^2 \leq c_{cc}^{VSC} \cdot l_c^{VSC} \quad c \in \mathcal{N}^{VSC} \tag{2.31b}$$
$$c_{ss}^{VSC} = (v_{s,ref}^{VSC})^2, \quad s \in \mathcal{N}^{VSC} \tag{2.32a}$$
$$q_s^{VSC} = q_{s,ref}^{VSC}, \quad s \in \mathcal{N}^{VSC} \tag{2.32b}$$

Goal:

$$\min \sum_i \left\{ c_{i,2}^{AC} \left(p_{i,gen}^{AC} \right)^2 + c_{i,1}^{AC} p_{i,gen}^{AC} + c_{i,0}^{AC} + e_{i,2}^{AC} \left(p_{i,res}^{AC} \right)^2 + e_{i,1}^{AC} p_{i,res}^{AC} + e_{i,0}^{AC} \right\}, \quad i \in \mathcal{N}^{AC} \tag{2.33}$$

- **AC Constraints**: ✓(2.17a)-(2.17g) consist of the second-order cone (SOC) relaxed power flow constraints [4]. ✓(2.18a)-(2.18b) are nodal power balance equations. ✓(2.19a)-(2.19b) are generator power output limits. (2.20) is the squared voltage magnitude limit. ✓(2.21a)-(2.21b) consists of the approximated power output constraint of RESs. Specifically, (2.21b) depicts the use of polygonal approxmiation to represent the originally circular feasible power region.

- **MTDC Constraints**: ✓(2.22a)-(2.22d) consist of the SOC relaxed power flow constraints [5]. ✓(2.23) is the squared voltage magnitude limit. ✓(2.24a)-(2.24c) represents the VSC control mode at DC terminal. Especially, the term v_{ij}^{MTDC} is replaced by the expression with respect to u_{ij}^{MTDC} in (2.24c), through Talyor expansion. Note that for (2.24a)-(2.24c), only one of them holds depending on the chosen control model.

- **VSC Constraint**: ✓(2.25a)-(2.25b) are the active and reactive power couplings between the AC grid and VSC. ✓(2.26a)-(2.26e) consist of the SOC relaxed active and reactive power injections from the view of node s.✓(2.27a)-(2.27e) consist of the SOC relaxed active and reactive power injections from the view of node c. ✓(2.28) is the reactive power injection from the view of node f. ✓(2.29) is the active power coupling between the MTDC grid and VSC. ✓(2.30a)-(2.30c) are related to the converter loss function. ✓(2.31a)-(2.31b) consist of the SOC relaxed voltage and current relationship from the view of node c. ✓(2.32a)-(2.32b) represent the VSC control mode at AC PCC. Note that only one of them is hold depending on the chosen control model.
- **Goal**: ✓(2.33) represent minimizing power generation costs.

2.4 Voltage phase recovery

For second-order cone relaxations in AC grid power flow or VSC AC-side power flow, the bus voltage phase is eliminated. However, the voltage phase is important in harmonic stability analysis of AC/DC hybrid power systems. Therefore, it is necessary to recover the system-wide voltage phase from the obtained AC/DC OPF results. Considering the complex AC nodal voltage is $\dot{v}_i^{AC} = v_i^{AC} e^{\mathrm{j}\theta_i^{AC}}$, the nonlinear coupling appears in power flow equations, such that:

$$\dot{v}i^{AC}\left(\dot{v}j^{AC}\right)^* = v_i^{AC} v_j^{AC}\left(\cos\delta ij^{AC} + \mathrm{j}\sin\delta ij^{AC}\right). \tag{2.33}$$

where $\delta_{ij}^{AC} = \theta_i^{AC} - \theta_j^{AC}$. To convexify the OPF model, SOC relaxation substitutes that $c_{ij}^{AC} = v_i^{AC} v_j^{AC}\cos\delta_{ij}^{AC}$ and $s_{ij}^{AC} = v_i v_j \sin\delta_{ij}^{AC}$. Thus avoiding explicit use of θ_i^{AC}. According to 2.33, we can see that the optimized variables satisfy:

$$\delta_{ij}^{AC} = \arctan 2\left(s_{ij}^{AC}, c_{ij}^{AC}\right), \tag{2.34}$$

thus δ_{ij}^{AC}, the AC branch phase difference, can be directly obtained based on s_{ij}^{AC} and c_{ij}^{AC}. Let $\mathcal{G}^{AC} := \left(\mathcal{N}^{AC}, \varepsilon^{AC}\right)$ denote the AC network graph. If we choose a slack bus with reference phase 0, then all bus phases can be recovered, along any spanning tree of $\mathcal{G}^{AC}$, such that:

$$\theta_j^{AC} = \theta_i^{AC} + \delta_{ij}^{AC}, \tag{2.35}$$

Then, we continue to consider that a VSC is connected to AC grid bus j. In this case, the PCC node s voltage phase can be directly obtained and regarded as reference voltage. Following the approach same to (2.33)-(2.35), the VSC AC-side voltage phase can be calculated.

3 Code Implementation

3.1 Data preparation

The data required for AC/DC OPF includes the fundamental data of the interconnected AC grids and the fundamental data of the MTDC grid, which are listed and shown in Fig. 3.1. All data are stored in .CSV files due to its compatibility with various language environments.

Users can manually fill in power grid parameters into respective .CSV files based on their specific needs. Below we present the details regarding the respective .CSV. The style of the saved data in .CSV is basically consistent with mpc structure used in Matpower[3] and MatACDC[4].

[3]MATPOWER Manual
[4]MatACDC Manual

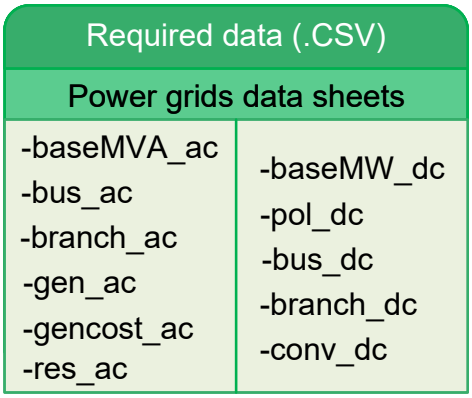


Figure 3.1: Required power grid data sheet

For the DC grid, the required fundamental data files includes `baseMW_dc.csv`, `pol_dc.csv`, `bus_dc.csv`, `branch_dc.csv`, `branch_dc.csv`. In the file, each column of the data corresponds to a parameter item. A specific rule needs to be followed when saving data into these files. The details about parameter items and descriptions of DC grid data files can be found in Table 1, 2, 3, 4, 5. Note that in `bus_dc.csv` and `branch_dc.csv` files, some parameter items may appear to be meaningless but serve as placeholders to maintain the same format style with the well-known Matpower.

For the AC grid, the required fundamental data files includes `baseMVA_ac.csv`, `bus_ac.csv`, `branch_ac.csv`, `gen_ac.csv`, `gencost_ac.csv`, `res_ac.csv`. The details about parameter items and descriptions of AC grid data files can be found in Table 6, 7, 8, 9, 10, 11. Matpower provides various AC grid case files. Sometimes, users may want to directly select and integrate multiple AC grids without manually modifying data files. To facilitate this process, we provide auxiliary functions `merge_ac` and `save_csv`. User can build up Matlab script as below.

```
mpc_merged = merge_ac('case9','case14'); % merge_ac('case_a','case_b','...')
% If no path is provided, saved in the current path.
save_csv(mpc_merged, directory);
```

After running the script, users can name the prefix of the series of .CSV filenames related to ac grids and directly type it. Finally, save information is shown in command window, like blow.

```
>> Reordered bus IDs for case9
>> Reordered bus IDs for case14
>> Enter a prefix for the CSV filenames: ac9ac14
>> Merged AC grid data has already been saved to:
>> D:\acdcopf\Tests\ac9ac14_baseMVA_ac.csv
>> D:\acdcopf\Tests\ac9ac14_bus_ac.csv
>> D:\acdcopf\Tests\ac9ac14_gen_ac.csv
>> D:\acdcopf\Tests\ac9ac14_branch_ac.csv
>> D:\acdcopf\Tests\ac9ac14_gencost_ac.csv
```

Column	Parameters	Description [Unit]
1	baseMW	Base power capacity [MW].

Table 1: baseMW_dc.csv file

Column	Parameters	Description [Unit]
1	pol	Number of poles (1=monopolar grid, 2=bipolar grid) [-].

Table 2: pol_dc.csv file

Column	Parameters	Description [Unit]
1	bus_i	Bus number [-].
2	—	N/A.
3	Pd	Active power loads [MW].
4	—	N/A.
5	—	N/A.
6	—	N/A.
7	—	N/A.
8	—	N/A.
9	—	N/A.
10	baseKV	Bus voltage level [kV].
11	—	N/A.
12	Vmax	Maximum bus voltage magnitude [p.u.].
13	Vmin	Minimum bus voltage magnitude [p.u.].

Table 3: bus_dc.csv file

Column	Parameters	Description [Unit]
1	fbus	From bus index [-].
2	tbus	To bus index [-].
3	r	Branch resistance [p.u.].
4	—	N/A.
5	—	N/A.
6	—	N/A.
7	—	N/A.
8	—	N/A.
9	—	N/A.
10	—	N/A.
11	—	N/A.
12	—	N/A.
13	—	N/A.

Table 4: branch_dc.csv file

Column	Parameters	Description [Unit]
1	busdc_i	Converter-connected DC bus index [-].
2	busac_i	Converter-connected AC bus index [-].
3	gridac	AC grid number of which converter belongs [-].
4	type_dc	Converter DC-side control modes (1=Constant DC power control, 2=Constant DC voltage control, 3=DC droop control) [-].
5	type_ac	Converter AC-side control modes (1=Constant AC voltage control, 2=Constant reactive power control) [-].
6	P_g	Setting of the active power output of converter DC terminal (applicable to the constant DC power control) [MW].
7	Q_g	Setting of the reactive power output of converter PCC bus (applicable to the constant reactive power control [MVAr].
8	Vtar	Setting of the voltage amplitude of converter DC terminal (applicable to the constant DC voltage control) [p.u.].
9	rtf	Resistance of the converter transformer [p.u.].
10	xtf	Reactance of the converter transformer [p.u.].
11	bf	Conductance of the converter filter [p.u.].
12	rc	Resistance of the converter Phase reactor [p.u.].
13	xc	reactance of the converter Phase reactor [p.u.].
14	basekVac	Base voltage of the converter AC side [p.u.].
15	Vmmax	Maximum voltage magnitude of the converter AC side [p.u.].
16	Vmmin	Minimum voltage magnitude of the converter AC side [p.u.].
17	Imax	Maximum current magnitude of the converter [p.u.].
18	status	Converter status (1=in service, 0=out of service) [p.u.].
19	LossA	Constant loss coefficient [MW].
20	LossB	Linear loss coefficient [kV].
21	LossCR	Rectifier quadratic loss coefficient [Ω].
22	LossCI	Inverter quadratic loss coefficient [Ω].

Table 5: conv_dc.csv file

Column	Parameters	Description [Unit]
1	baseMVA	Base power capacity [MVA].

Table 6: baseMVA_ac.csv file

Column	Parameters	Description [Unit]
1	bus_i	Bus number [-].
2	type	Bus type (3=Slack, 2=PV, 1=PQ) [-]
3	Pd	Active power loads [MW].
4	Qd	Reactive power loads [MVAr].
5	Gs	Shunt conductance (at V=1.0 p.u.) [MW].
6	Bs	Shunt susceptance (at V=1.0 p.u.) [MVAr].
7	area	Area number [-].
8	Vm	Reactive power loads [p.u.].
9	Va	Reactive power loads [drgrees].
10	baseKV	Bus voltage level [kV].
11	zone	Loss zone [-].
12	Vmax	Maximum bus voltage magnitude [p.u.].
13	Vmin	Minimum bus voltage magnitude [p.u.].

Table 7: bus_ac.csv file

Column	Parameters	Description [Unit]
1	fbus	From bus index [-].
2	tbus	To bus index [-].
3	r	Branch resistance [p.u.].
4	x	Branch reactance [p.u.].
5	b	Line charging susceptance [p.u.].
6	rateA	Rate A (MVA limit for normal operation) [MVA].
7	rateB	Rate B (MVA limit for short-term) [MVA].
8	rateC	Rate C (MVA limit for emergency) [MVA].
9	ratio	Transformer tap ratio (if applicable) [-].
10	angle	Transformer phase shift angle [degrees].
11	status	Line status (1=active, 0=out-of-service) [-].
12	angmin	Minimum voltage angle difference [degrees].
13	angmax	Maximum voltage angle difference [degrees].

Table 8: branch_ac.csv file

Column	Parameters	Description [Unit]
1	bus	Generator's bus number [-].
2	Pg	Active power output [MW].
3	Qg	Reactive power output [MVAr].
4	Qmax	Maximum reactive power output [MVar].
5	Qmin	Minimum reactive power output [MVar].
6	Vg	Voltage magnitude setpoint [p.u.].
7	mBase	Total MVA base of generator [MVA].
8	status	Generator status (1=in service, 0=out of service) [-].
9	Pmax	Maximum active power output [MW].
10	Pmin	Minimum active power output [MW].
11	Pc1	lower real power output of PQ capability curve [MW].
12	Pc2	upper real power output of PQ capability curve [MW].
13	Qc1min	Minimum reactive power output at Pc1 [MVar].
14	Qc1max	Maximum reactive power at Pc1 [MVar].
15	Qc2min	Minimum reactive power at Pc2 [MVar].
16	Qc2max	Maximum reactive power at Pc2 [MVar].
17	ramp_agc	Ramp rate for automatic generation control [MW/min].
18	ramp_10	Ramp rate for 10-minute reserves [MW].
19	ramp_30	Ramp rate for 30-minute reserves [MW].
20	ramp_q	Ramp rate for reactive power [MVar/min].
21	apf	Area participation factor [-].

Table 9: gen_ac.csv file

Column	Parameters	Description [Unit]
1	bus	Cost function type(1=piecewise, 2=polynomial) [-].
2	startup	Startup cost [$].
3	shuntdown	Shutdown cost [$].
4	n	Number of data points (if piecewise linear) or order of polynomial [$].
5	cost	Coefficients of polynomial (from hgithest to lowest order) or piecewise data points.

Table 10: gencost_ac.csv file

Column	Parameters	Description [Unit]
1	bus	Generator's bus number [-].
2	Presmax	Maximum available active power [MW].
3	Sresmax	Maximum apparent power [MVA].
4	bus	Cost function type (1=piecewise, 2=polynomial) [-].
5	startup	Startup cost [$].
6	shuntdown	Shutdown cost [$].
7	n	Number of data points (if piecewise linear) or order of polynomial [$].
8	cost	Coefficients of polynomial (from highest to lowest order) or piecewise data points.

Table 11: res_ac.csv file

3.2 Parameter processing

Language-specific functions `create_ac` and `create_dc` reads .csv file data related to the interconnected AC grids and the MTDC grid. Depending on the programming environment, different handling approaches are used to read and organize these .csv files, as listed in Table 12.

Table 12: .CSV File Data Handling Approaches Across Programming Languages

Language	Read .csv Files	Transformed Data Structure
Matlab	`readmatrix` (built-in)	Uses a `struct` where each field name maps to a numeric matrix or a scalar.
Python	`pandas.read_csv`	Uses a `dict` where each string key maps to a `NumPy` array or a scalar.
Julia	`CSV.File` + `DataFrame`	Uses a `Dict{String, Any}` where each string key maps to a numeric array or a scalar.
Cpp	`ifstream` + `csv_reader` (custom)	Uses an `unordered_map<std::string, Eigen::MatrixXd>` where each string key maps to an `Eigen` matrix.

Subsequently, language-specific functions `params_ac` and `params_dc` are further utilized to extract the essential and critical parameters required for constructing the AC/DC OPF model. During this process, different language-specific conventions are adopted to define one-dimensional, two-dimensional, and three-dimensional parameters, which are listed in Table 13.

Table 13: Parameter Defining Across Programming Languages

Language	1D Parameter (`fbus_dc`, `nbuses_ac`, etc.)	2D Parameter (`bus_dc`, `pd_ac`, etc.)	3D Parameter (`bus_entire_ac`, etc.)
Matlab	`vector`/`x(i)`, `cell`/`x{n}`	`matrix`/`x(i, j)`, `cell`/`x{n}(i)`	`cell`/`x{n}(i, j)`
Python	`np.ndarray`/`x[i]`, `list`/`x[n]`	`np.ndarray`/`x[i, j]`, `list`/`x[n][i]`	`list`/`x[n][i, j]`
Julia	`Vector`/`x[i](x[n])`	`Matrix`/`x[i, j]`, `Vector`/`x[n][i]`	`Vector`/`x[n][i, j]`
Cpp	`Eigen::Vector`/`x(i)`, `std::vector`/`x[i]`	`Eigen::Matrix`/`x(i, j)`, `std::vector`/`x[n](i)`	`std::vector`/`x[n](i, j)`

★ The mark means `type`/`syntax`

3.3 Variable definition

The core OPF solver can be found in `solve_opf`, where the AC/DC OPF model is formulated and solved. **ACDC-OpFlow** formulates AC/DC OPF model as a SOCP problem, which has been presented in Section 2.3.

In implementations across Matlab, Python, Julia, and Cpp, **ACDC-OpFlow** utilizes Gurobi, a powerful convex optimization solver that supports SOCP solving and offers academic licenses free of charge for research and educational use[5], as the unified solver backend. To facilitate AC/DC OPF modelling and solver invocation, **ACDC-OpFlow** integrates each language with a widely adopted modelling framework and corresponding solver interface. Table 14 summarizes the dependencies used for optimization in each programming environment.

Table 14: Optimization Modeling Dependencies Across Programming Languages

Language	**Dependencies (functionalities)**
Matlab	YALMIP[6] (modeling interface + solver bridge)
Python	Pyomo[7] (modeling interface) + gurobipy (solver bridge)
Julia	JuMP[8] (modeling interface) + Gurobi.jl (solver bridge)
Cpp	Gurobi C++ API[9] (modeling interface + solver interface)

To further support the optimization modeling, each implementation explicitly defines optimization decision variables in compatible formats with each modeling framework, as shown in Table 15.

Table 15: Optimization Variable Definition Across Languages

Language	**Define Variables**
Matlab	`sdpvar(...)`
Python	`Var(..., domain = Reals)`
Julia	`@variable(model, ...)`
Cpp	`model.addVar(..., GRB_CONTINUOUS)`

Similar to parameter definition, optimization variable definition also conforms to language-specific conventions, and the summary of representative expressions for defining one-dimensional, two-dimensional, and three-dimensional variables in each language environment are listed below:

```
%% Variable Defining in Matlab via YALMIP

% 1D Variable (e.g., vn2_dc)
  vn2_dc = sdpvar(nbuses_dc, 1);
% 2D Variable (e.g., pij_dc)
  pij_dc = sdpvar(nconvs_dc, nconvs_dc, 'full');
% 2D Variable (e.g., vn2_ac)
  vn2_ac = cell(ngrids, 1);
  for ng = 1:ngrids
```

[5] https://www.gurobi.com/academia/academic-program-and-licenses/
[6] https://yalmip.github.io/
[7] https://www.pyomo.org/
[8] https://jump.dev/
[9] https://docs.gurobi.com/projects/optimizer/en/current/reference/cpp.html

```
      vn2_ac{ng} = sdpvar(nbuses_ac{ng}, 1);
end
% 3D Variable (e.g., pij_ac)
  pij_ac = cell(ngrids, 1);
  for ng = 1:ngrids
      pij_ac{ng} = sdpvar(nbuses_ac{ng}, nbuses_ac{ng}, 'full');
end
```

```
# Variable definition in Python via Pyomo

# 1D variable (e.g., vn2_dc)
  model.vn2_dc = Var(range(nbuses_dc), domain=Reals)
# 2D variable (e.g., pij_dc)
  model.pij_dc = Var(range(nbuses_dc), range(nbuses_dc), domain=Reals)
# 2D variable (e.g., vn2_ac)
  model.vn2_ac = Var(
      [(ng, i) for ng in range(ngrids) for i in range(nbuses[ng])],
      domain=Reals)
# 3D variable (e.g., pij_ac)
  model.pij_ac = Var(
      [(ng, i, j) for ng in range(ngrids) for i in range(nbuses[ng]) for j in
      ↪  range(nbuses[ng])],
      domain=Reals)
```

```
# Variable Definition in Julia via JuMP

# 1D Variable (e.g., vn2_dc)
  vn2_dc = @variable(model, [1:nbuses_dc])
# 2D Variable (e.g., pij_dc)
  @variable(model, pij_dc[1:nbuses_dc, 1:nbuses_dc])
# 2D Variable (e.g., vn2_ac)
  vn2_ac = Vector{Vector{JuMP.VariableRef}}(undef, ngrids)
  for ng in 1:ngrids
      vn2_ac[ng] = @variable(model, [1:nbuses_ac[ng]])
  end
# 3D Variable (e.g., pij_ac)
  pij_ac = Vector{Matrix{JuMP.VariableRef}}(undef, ngrids)
  for ng in 1:ngrids
      pij_ac[ng] = @variable(model, [1:nbuses_ac[ng], 1:nbuses_ac[ng]])
  end
```

```
// Variable Definition in Cpp via Gurobi Cpp API

// 1D Variable (e.g., vn2_dc)
  Eigen::Matrix<GRBVar, Eigen::Dynamic, 1> vn2_dc(nbuses_dc);
  for (int i = 0; i < nbuses_dc; ++i) {
```

```
      vn2_dc(i) = model.addVar(-GRB_INFINITY, GRB_INFINITY, 0.0, GRB_CONTINUOUS);
  }
// 2D Variable (e.g., pij_dc)
  Eigen::Matrix<GRBVar, Eigen::Dynamic, Eigen::Dynamic> pij_dc(nbuses_dc, nbuses_dc);
  for (int i = 0; i < nbuses_dc; ++i) {
      for (int j = 0; j < nbuses_dc; ++j) {
          pij_dc(i, j) = model.addVar(-GRB_INFINITY, GRB_INFINITY, 0.0,
          ↪ GRB_CONTINUOUS);
      }
  }
// 2D Variable (e.g., vn2_ac)
  std::vector<Eigen::Matrix<GRBVar, Eigen::Dynamic, 1>> vn2_ac(ngrids);
  vn2_ac[ng].resize(nbuses_ac[ng]);
  for (int i = 0; i < nbuses_ac[ng]; ++i) {
      vn2_ac[ng](i) = model.addVar(-GRB_INFINITY,  GRB_INFINITY, 0.0, GRB_CONTINUOUS);
  }
// 3D Variable (e.g., pij_ac)
  std::vector<Eigen::Matrix<GRBVar, Eigen::Dynamic, Eigen::Dynamic>> pij_ac(ngrids);
  pij_ac[ng] = Eigen::Matrix<GRBVar, Eigen::Dynamic, Eigen::Dynamic>(nbuses_ac[ng],
  ↪ nbuses_ac[ng]);
  for (int i = 0; i < nbuses_ac[ng]; ++i) {
      for (int j = 0; j < nbuses_ac[ng]; ++j) {
          pij_ac[ng](i, j) = model.addVar(-GRB_INFINITY, GRB_INFINITY, 0.0,
          ↪ GRB_CONTINUOUS);
      }
  }
```

3.4 Constraint construction

Each implementation explicitly adds operational constraints in compatible formats with each modeling framework, as shown in Table 16. Next, we will take the addition of SOC-based DC power flow constraints as an example to present code snippets under different implementations more specifically.

Table 16: Constraint Construcution Across Languages

Language	Constraints Construction
Matlab	`Cons = [Cons; expr]`
Python	`model.Constraint = Constraint(expr)`
Julia	`@constraint(model, expr)`
Cpp	`model.addConstr(expr)` (for linear constraints), `model.addQConstr(expr)` (for SOC constraints)

```m
%% Constraint Construction in Matlab via YALMIP, with vectorized formulation.

% --- DC Power Flow Constraints (Second-Order Cone Relaxation) ---
zij_dc = 1./abs(ybus_dc);
zij_dc = zij_dc-diag(diag(zij_dc));
zij_dc(isinf(zij_dc)) = 1e4;

con_dc = [con_dc;
    pn_dc == sum(pij_dc, 2) * pol_dc;
    pij_dc + pij_dc' == zij_dc .* lij_dc;
    repmat(vn2_dc, 1, nbuses_dc) - repmat(vn2_dc', nbuses_dc, 1) == zij_dc .* (pij_dc - pij_dc');
    pij_dc.^2 <= lij_dc .* repmat(vn2_dc, 1, nbuses_dc);
    vn2_dc >= 0;
    lij_dc >= 0];
```

```python
# Constraint Construction in Python via Pyomo, with for-loop formulation.

model.addconstraints = ConstraintList()
# ------------------------------
# DC Power Flow Constraints (Second-Order Cone Relaxation)
# ------------------------------
zij_dc = 1.0 / np.abs(ybus_dc.toarray())
zij_dc = zij_dc - np.diag(np.diag(zij_dc))
zij_dc[np.isinf(zij_dc)] = 1e4

for i in range(nbuses_dc):
    model.addconstraints.add(model.pn_dc[i]  == pol_dc * sum(model.pij_dc[i, j] for j in range(nbuses_dc)))
    model.addconstraints.add(model.vn2_dc[i] >= 0)
    for j in range(nbuses_dc):
        model.addconstraints.add(model.pij_dc[i, j] + model.pij_dc[j, i] == zij_dc[i, j] * model.lij_dc[i, j])
        model.addconstraints.add(model.vn2_dc[i] - model.vn2_dc[j] == zij_dc[i, j] * (model.pij_dc[i, j] - model.pij_dc[j, i]))
        model.addconstraints.add(model.pij_dc[i, j]**2 <= model.lij_dc[i, j] * model.vn2_dc[i])
        model.addconstraints.add(model.lij_dc[i, j] >= 0)
```

```julia
# Constraint Construction in Julia via JuMP, with vectorized formulation.

# ------------------------------
# DC Power Flow Constraints (Second-Order Cone Relaxation)
# ------------------------------
zij_dc = 1.0 ./ abs.(ybus_dc)
zij_dc = zij_dc .- Diagonal(diag(zij_dc))
zij_dc[isinf.(zij_dc)] .= 1e4
```

```julia
@constraint(model, pn_dc .== pol_dc * sum(pij_dc, dims=2))
@constraint(model, pij_dc .+ pij_dc' .== zij_dc .* lij_dc)
@constraint(model, vn2_dc .- vn2_dc' .== zij_dc .* (pij_dc .- pij_dc'))
@constraint(model, pij_dc.^2 .<= lij_dc .* repeat(vn2_dc, 1, nbuses_dc))
@constraint(model, vn2_dc .>= 0)
@constraint(model, lij_dc .>= 0)
```

```cpp
// Constraint Construction in Cpp via Gurobi Cpp API, with for-loop formulation.

// 1. Constrains for dc power flow - second-order cone relaxation
Eigen::MatrixXd y_dc_dense = Eigen::MatrixXd(y_dc);
Eigen::MatrixXd zij_dc = y_dc_dense.array().abs().cwiseInverse().matrix();
zij_dc -= zij_dc.diagonal().asDiagonal();
for (int i = 0; i < zij_dc.rows(); ++i) {
    for (int j = 0; j < zij_dc.cols(); ++j) {
        if (std::isinf(zij_dc(i, j))) {
            zij_dc(i, j) = 1e4;
        }
    }
}

for (int i = 0; i < nbuses_dc; ++i) {
    GRBLinExpr power_flow_sum = 0.0;
    for (int j = 0; j < nbuses_dc; ++j) {
        power_flow_sum += pij_dc(i, j);
    }
    model.addConstr(pn_dc(i) == power_flow_sum * pol_dc);
}

for (int i = 0; i < nbuses_dc; ++i) {
    for (int j = 0; j < nbuses_dc; ++j) {
        model.addConstr(pij_dc(i, j) + pij_dc(j, i) == zij_dc(i, j) * lij_dc(i, j));
        model.addQConstr(pij_dc(i, j) * pij_dc(i, j) <= lij_dc(i, j) * vn2_dc(i));
        model.addConstr(vn2_dc(i) - vn2_dc(j) == zij_dc(i, j) * (pij_dc(i, j) -
        ↪  pij_dc(j, i)));
    }
}

for (int i = 0; i < nbuses_dc; ++i) {
    model.addConstr(vn2_dc(i) >= 0.0);
    for (int j = 0; j < nbuses_dc; ++j) {
        model.addConstr(lij_dc(i, j) >= 0.0);
        }
}
```

4 Examples

4.1 Running a case

Matlab, Python, Julia, and Cpp follows a unified running process. As mentioned earlier, before running the AC/DC OPF, we need to prepare the required .CSV files to save the parameter data related to interconnected AC grids and the MTDC grid. In each programming language, users can execute AC/DC OPF through the code snippet as below.

```
solve_opf('mtdc3slack_a', 'ac9ac14',
          'vscControl', true,
          'writeTxt', false,
          'plotResult', true)
```

Noted that in `solve_opf()`, besides attributes `'matdc3slack_a'` and `'ac9ac14'`, which respectively represent the case name of MTDC grid and interconnected AC grids, there are three additional attributes: `'vscControl'`, `'writeTxt'`, and `'plotResult'` need to be set:

- `'vscControl'` is used to determine if the VSC control constraints formulated as (2.24a)-(2.24c) and (2.32a)-(2.32b) are activated. Its default setting is `true` means the VSC control constraints need to be added into the formed AC/DC OPF model, and the control settings are pre-defined in the imported `conv_dc.csv` file. When the VSC control mode is considered to be optimized, it should not be pre-specified. `'vscControl'` should be set to `false`, which means that (2.24a)-(2.24c) and (2.32a)-(2.32b) are excluded from the formed AC/DC OPF model.

- `'writeTxt'` is used to determine whether the printed AC/DC OPF results shown in Fig. 4.1 should be saved as a text file. By default, it is set to `false`, and no .txt file will be generated. If set to `true`, a .txt file named `opf_result.txt` will be produced in the current working directory. Regardless of whether `'writeTxt'` is set to `true` or `false`, the textual results shown in Fig. 4.1 will always be printed to the terminal window.

Figure 4.1: Printed AC/DC OPF text.

```
================================================================================================
|     AC Grid Bus Data                                                                         |
================================================================================================
 Area   Branch  Voltage        Generation              Load                    RES
 #      #       Mag [pu]    Pg [MW]   Qg [MVAr]   Pd [MW]   Qd [MVAr]   Pres [MW]  Qres [MVAr]
-----  -------  --------   --------   ---------   -------   ---------   ---------  -----------
   1       1     1.046*    87.560     118.622      0.000       0.000        -           -
   1       2     1.094    134.532     299.997      0.000       0.000        -           -
   1       3     1.037     93.126      91.777      0.000       0.000        -           -
   1       4     1.009        -           -        0.000       0.000        -           -
   1       5     1.000        -           -       90.000      30.000     40.000       -8.082
   1       6     1.016        -           -        0.000       0.000        -           -
   1       7     1.002        -           -      100.000      35.000        -           -
   1       8     1.015        -           -        0.000       0.000        -           -
   1       9     0.983        -           -      125.000      50.000        -           -
   2       1     1.053*   168.897       5.726      0.000       0.000     35.000        8.989
   2       2     1.027     32.396      26.193     21.700      12.700        -           -
   2       3     0.990      0.000      25.297     94.200      19.000        -           -
   2       4     0.997        -           -       47.800      -3.900        -           -
   2       5     1.000        -           -        7.600       1.600        -           -
   2       6     1.042      0.000      21.712     11.200       7.500        -           -
   2       7     1.034        -           -        0.000       0.000        -           -
```

```
  2      8     1.060      0.000     15.435      0.000       0.000        -          -
  2      9     1.028        -         -        29.500      16.600       -          -
  2     10     1.023        -         -         9.000       5.800       -          -
  2     11     1.029        -         -         3.500       1.800       -          -
  2     12     1.027        -         -         6.100       1.600       -          -
  2     13     1.022        -         -        13.500       5.800       -          -
  2     14     1.007        -         -        14.900       5.000       -          -
-----   -----   --------   --------   ---------   -------   ---------   ---------  -----------
The total generation cost is $10815.00/MWh(€10013.89/MWh)

... ... (omitted)

================================================================
|     MTDC Branch Data                                         |
================================================================
Branch  From   To     From Branch    To Branch      Branch Loss
#       Bus#   Bus#   Flow Pij [MW]  Flow Pij [MW]  Pij_loss [MW]
------  -----  -----   ---------      ---------      --------
    1      1      2      31.986        -31.724         0.262
    2      2      3       7.281         -7.268         0.014
    3      1      3      28.014        -27.732         0.282
------  -----  -----   ---------      ---------      --------
The totoal DC network losses is 0.557 MW .

Execution time is 5.295s .
```

- `'plotResult'` is used to determine whether the visualization of AC/DC OPF results, as shown in Fig. 4.2, is enabled. By default, it is set to `true`, and the corresponding plots will be generated automatically. Such visualizations provide intuitive insights into system-wide power flow. In some cases, the system size is relatively large, and the visualization may become cluttered due to the excessive amount of information displayed. Fortunately, the plotting tools used in each programming languages are interactive and support zooming, allowing users to focus on specific regions of interest. Of course, if the user is not interested in visualization, setting `'plotResult'` to `false` will disable plotting outputs.

4.2 Benchmark

4.2.1 Performance comparison

To compare programming languages, benchmark scripts were developed. The test cases consistently used the same three-terminal MTDC grid case `'mtdc3slack'` a combined with four representative interconnected AC grid cases from small to large scales that are: `'ac9ac14'`, `'ac14ac57'`, `'ac57ac118'`, and `'ac118ac300'`. For all experiments, each was executed five times per test case and with three warmup trial runs to avoid the effect of just-in-time (JIT) compilation and delayed code execution due to memory caching.

Execution time and memory allocation were stored for each execution, and a post-execution process was carried out to compute median values from the results. In the case of Julia, run time was recorded using the `@elapsed` macro provided in the `BenchmarkTools.jl` package, while memory allocations were tracked at the systems level by observing process monitoring. Julia utilized a separate, although framework-based dictionary, dependent on grid modeling but also leveraged modular, very high-performance versions of the algorithms. For Python runtime, `time.time()` was used and memory by using the `psutil` library to return the resident set size (RSS). The Python benchmark script was made to accommodate missing input files, should there be a case of non-completion.

Cpp performance benchmarks used the high-resolution `std::chrono` clock for timing, while memory measurements came from `GetProcessMemoryInfo()` on Windows. Cpp was also in an

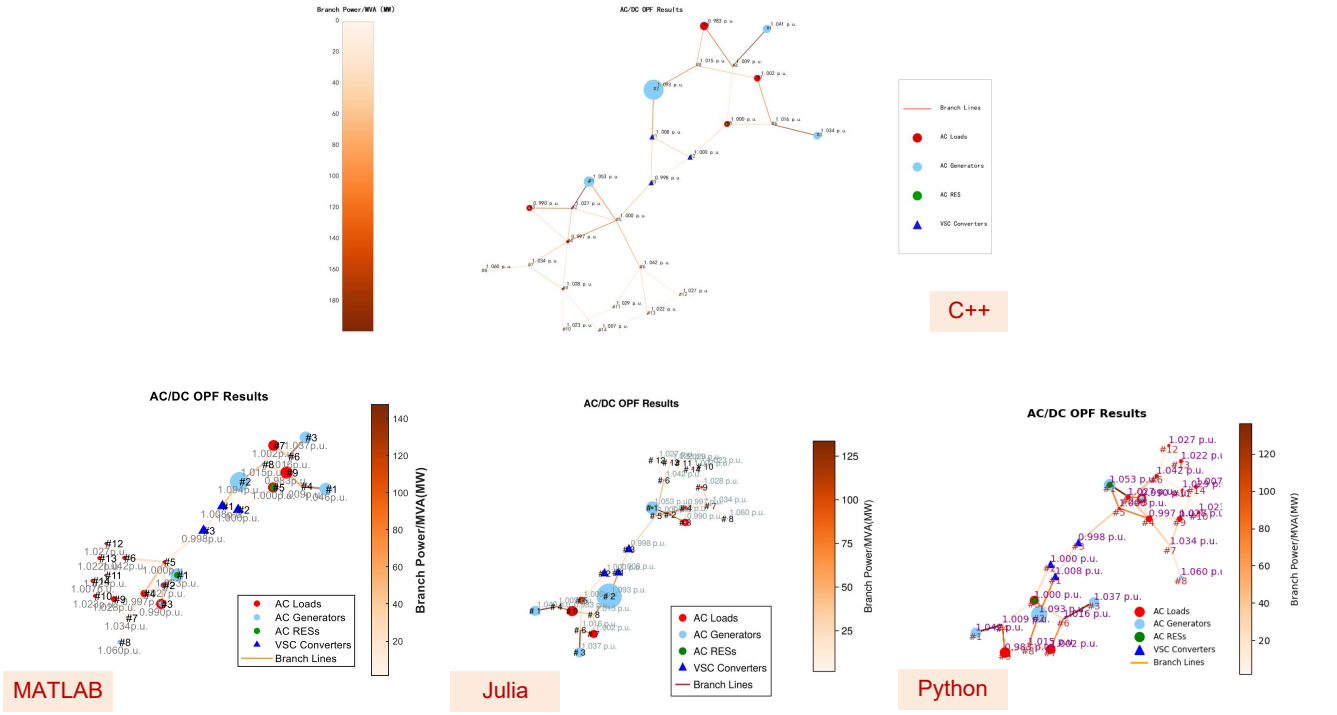


Figure 4.2: Visualization of AC/DC OPF results across programming languages. The size of ● is proportional to the AC load level. The size of ● is proportional to the AC generator power level. The size of ● is proportional to the AC RES power level. The color intensity of ▭ indicates the branch power level.

object-oriented context with custom memory stuff to reduce dynamic allocations. Matlab used `tic` and `toc` to measure execution time and had a custom `et_memory_usage()` method to track memory and invoke garbage collection every run. To make sure memory use was consistent every run, the benchmark had to invoke a pause for a few seconds to get consistent memory states. All implementations run in the same benchmarking environment to give a fair and consistent comparison across languages. The statistics collected, therefore, included median run time, run time variation (min-max), peak memory usage, and relative memory use normalized against the smallest test case as the baseline for each language.

The benchmarking results are summarized in Table 17. For each case, we report the median execution time, run-time range, peak memory usage, and relative memory usage normalized to the smallest test case within each language.

4.2.2 Discussion

The results indicate striking variations in execution time and memory consumption between implementations.

Runtime:

- Across the benchmarked cases, Julia consistently had the fastest execution times. For example, in the largest case, `'ac118ac300'`, Julia was the only language to return a solution in seconds (4.446 s), outperforming Cpp (38.431 s), Python (35.976 s), and especially Matlab (not computed successfully within an hour). Julia's position was also aided by the use

Table 17: Benchmark Results Summary Across Programming Languages

Case	Language	Time (s)	Time Range (s)	Memory (MB)	Rel. Mem.
ac9ac14 +mtdc3slack_a	Matlab	7.673	7.611–7.732	0.1	1.0x
	Cpp	0.139	0.133–0.142	0.0	0x
	Julia	0.055	0.052–0.072	0.1	1.0x
	Python	0.138	0.127–0.182	1.7	17.0x
ac14ac57 +mtdc3slack_a	Matlab	157.353	154.715–166.027	5.3	0.14x
	Cpp	1.780	1.478–2.365	0.5	0.013x
	Julia	0.233	0.202–0.940	36.8	1.0x
	Python	1.014	0.997–1.054	2.4	0.07x
ac57ac118 +mtdc3slack_a	Matlab	177.273	92.259–245.281	14.9	0.12x
	Cpp	3.464	3.299–4.056	0.8	0.006x
	Julia	0.799	0.783–0.821	129.4	1.0x
	Python	5.063	4.977–5.240	2.6	0.02x
ac118ac300 +mtdc3slack_a	Matlab	-	-	-	-
	Cpp	38.431	37.126–38.924	0.5	0x
	Julia	4.446	4.353–5.343	537.7	1.0x
	Python	35.976	28.344–96.248	1.9	0.003x

★ The case ac118ac300+mtdc3slack_a is not computed successfully in MATLAB

of JIT compilation, type stability, and fast numerical computing libraries that are heavily optimized.

- Cpp had particularly strong and consistent performance, as it was often ranked second in execution time to Julia with sometimes very small margins. For example, Cpp solved `'ac14ac57'` in 1.780 s and `'ac57ac118'` in 3.464 s, indicating an ability to scale with the larger case. While Cpp may not be capable of the peak performance that Julia demonstrated, it still represented a large improvement over Matlab and Python, in particular, the larger cases.

- Python had more mixed performance. For some of the smaller cases like `'ac9ac14'`, it performed reasonably well (0.138 s, similar to Cpp) but for larger problems like `'ac57ac118'`, its performance suffered 5.063 s but still much better than Matlab). Overall, this suggests that Python could return competitive performance when appropriately optimized but also suffers from the overheads of being interpreted and dynamically typed languages.

- Matlab consistently had the slowest execution times, taking 7.673 s for `'ac9ac14'` and up to 177.273 s for `'ac57ac118'`. Although Matlab offers a high-level and user-friendly interface for numerical computing, it is not optimized for large-scale, real-time OPF problems, compared to compiled or JIT-compiled languages.

Memory Usage:

- Cpp had the lowest memory use in the benchmarks, achieving 0.0MB on `'ac9ac14'` and only 0.8 MB on `'ac57ac118'`. Cpp's efficiency in memory usage can be attributed to its manual control and low overhead, making it a great option for embedded or constrained environments.

- Matlab also achieved low memory usage of 0.1 MB or small cases, and up to 14.9 MB for `'ac57ac118'`. These values show that, while executing slowly, Matlab uses in-place operations and other efficient numerical operations to limit memory usage.

- Python had reasonable and moderate memory usage, between 1.7 MB on `'ac9ac14'` and 2.6 MB on `'ac57ac118'`. However, for the small case calls, its relative memory use was at the high end due likely to either run-time overhead from Python itself or memory overhead to include references to external libraries (i.e. `numpy`, or, `pandas` etc.).

- In the larger cases, Julia consumed the most memory during execution. For example, the largest call (`'ac57ac118'`) was finished using 129.4 MB, and the second largest (`'ac118ac300'`) finished with 537.7 MB. This likely reflects its design choice to have more memory used to enable faster execution speed and array "fusion". This is generally fine in high-performance computing, as long as the overall needs of memory consumption remain low and adequate overall.

Scalability:

- The scalability of Julia was noted to be very impressive since execution times increased sub-linearly, from 0.055 s for `'ac9ac14'` to 0.799 s for `'ac57ac118'`, and 4.446 s for `'ac118ac300'`, the largest case. This shows that Julia is very robust and is quite appropriate for large-scale AC/DC OPF problems or even for real-time applications.

- The scalability of Cpp was also very strong. Times grew from 0.139 s for `'ac9ac14'` to 3.464 s for `'ac57ac118'`, and 38.431 s for `'ac118ac300'`. This projection is enhanced because of the compile-time optimizations and low-level control Cpp enables to increase speed. However, with this comes a higher level of complexity to develop.

- The scalability for Python was reasonable. However, there was too much variability in run time as the size of the system increased—the largest case,`'ac57ac118'`, had running times range from 28.344 s to 96.248 s, which influenced background overhead and behaviour of the number of libraries.

- Beyond time, Matlab did not scale significantly. It was not able to complete the largest case `'ac118ac300'`, and the response time for the mid-size cases `'ac14ac57'` (157.353 s) and `'ac57ac118'` (177.273 s) had large increases in execution time. This demonstrates that Matlab is not suitable for large-scale optimization still making it a better alternative in prototyping scenarios and for educational purposes.

A Additional Discussion

A.1 Power flow accuracy

ACDC-OpFlow takes the convex relaxed power flow constraints for future model expansion. One major advantage is that it preserves the convexity of the OPF model, enabling the potential integration of DC topology reconfiguration as a tractable mixed-integer convex optimization problem. Naturally, the downside is a potential loss in power flow calculation accuracy. Taking Fig. A.1 as an example, we provide an intuitive comparison between the power flow results obtained using the standard power flow calculation method (from MATACDC) and those derived from the convex-scaled approach (from **ACDC-OpFlow**).

The comparison results are listed in Tables 18-26. We can observe that in SOC relaxed power flow models, the calculation of nodal voltage is generally more accurate than that of branch power flows. This is because SOC relaxations directly approximate the quadratic voltage magnitude relationships using convex constraints, which tend to preserve voltage-related variables more reliably. In contrast, branch power flows involve nonlinear combinations of voltages and angles, and their SOC-relaxed representations often introduce greater approximation errors [6],[7].

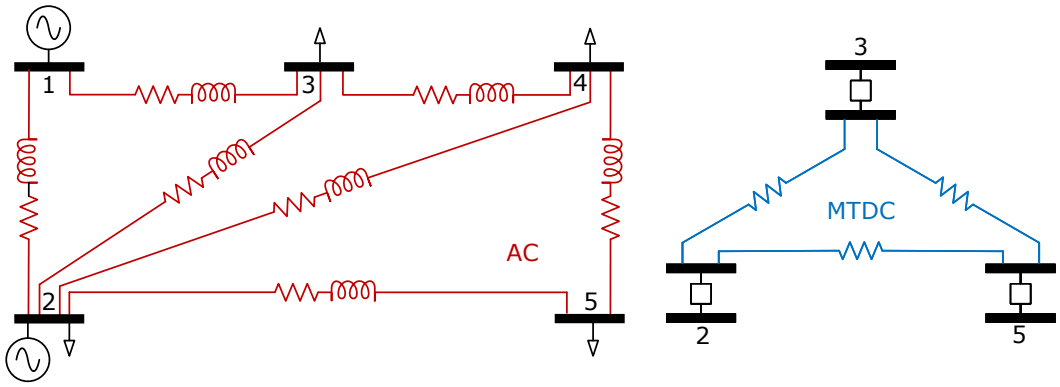


Figure A.1: Simple test system from MATACDC. The AC system is from `"case5_stagg"` and MTDC system is from `"case5_stagg_MTDCslack"`.

Table 18: Comparison of AC Node Voltage

AC Node	Node Voltage [p.u.]		Aboslute Error [p.u.]
	ACDC-OpFlow	MATACDC	
#1	1.06	1.06	0
#2	1	1	0
#3	1	1	0
#4	0.996	0.996	0
#5	0.99	0.991	0.001

Table 19: Comparison of DC Node Voltage

DC Node	Node Voltage [p.u.]		Aboslute Error [p.u.]
	ACDC-OpFlow	MATACDC	
#1	1.008	1.008	0
#2	1	1	0
#3	0.998	0.998	0

Table 20: Comparison of AC Branch Power

AC Branch	Branch Power [MW]		Aboslute Error [MW]
	ACDC-OpFlow	MATACDC	
#1	98.848	98.38	0.468
#2	34.699	35.26	0.561
#3	12.471	13.25	0.779
#4	16.504	17.08	0.576
#5	25.825	25.33	0.495
#6	24.339	23.09	1.249
#7	0.612	-0.07	0.682

Table 21: Comparison of DC Branch Power

DC Branch	Branch Power [MW]		Aboslute Error [MW]
	ACDC-OpFlow	MATACDC	
#1	31.986	30.66	1.326
#2	7.281	8.52	1.239
#3	28.014	27.96	0.054

Table 22: Comparison of AC Branch Power Loss

AC Branch	Branch Loss [MW]		Aboslute Error [MW]
	ACDC-OpFlow	MATACDC	
#1	2.730	2.717	0.013
#2	1.038	1.062	0.024
#3	0.103	0.116	0.013
#4	0.169	0.181	0.012
#5	0.267	0.257	0.01
#6	0.062	0.057	0.005
#7	0.004	0.004	0

Table 23: Comparison of DC Branch Power Loss

DC Branch	Branch Loss [MW]		Aboslute Error [MW]
	ACDC-OpFlow	MATACDC	
#1	0.262	0.24	0.022
#2	0.014	0.02	0.006
#3	0.282	0.28	0.002

Table 24: Comparison of VSC Converter Loss

VSC	Converter Loss [MW]		Aboslute Error [MW]
	ACDC-OpFlow	MATACDC	
#1	1.233	1.29	0.057
#2	1.124	1.14	0.016
#3	1.147	1.17	0.023

Table 25: Comparison of Generator Active Power

Generator	Active Power Generation [MW]		Aboslute Error [MW]
	ACDC-OpFlow	MATACDC	
#1	133.547	133.64	0.093
#2	40	40	0

Table 26: Comparison of Generator Reactive Power

Generator	Reactive Power Generation [Mvar]		Aboslute Error [Mvar]
	ACDC-OpFlow	MATACDC	
#1	84.333	84.32	0.013
#2	-32.11	-32.84	0.73

A.2 Integrate into Harmony

The AC/DC OPF models and solution procedures of **ACDC-OpFlow** have been integrated into **Harmony**, a framework for modeling and stability analysis of AC/MTDC power systems. In **Harmony** UI shown in Figure. A.2, a bunch of OPF example is provided. Users can select an example, run the corresponding AC/DC OPF calculation, and inspect the resulting OPF results as shown in Figure. A.3

This integration allows an OPF operating point to be passed to converter models for equilibrium calculation, linearization, and subsequent stability analysis within a common network representation. For standalone OPF studies, the comparisons in the preceding subsection demonstrate useful agreement in generation cost and generator dispatch for the tested systems. When the same solution is used to initialize a converter and its controls, additional quantities become important: PCC active and reactive powers, AC and DC voltages, and recovered voltage angles must also be consistent with the nonlinear device equations. Agreement in generation cost alone does not establish this consistency. The following strengthened formulation, denoted MISOCP-b, was therefore introduced in **Harmony**.

Model modifications

In the original nonlinear converter model, the current magnitude and its squared-current variable satisfy

$$l_c^{VSC} = (I_c^{VSC})^2, \qquad 0 \le I_c^{VSC} \le \overline{I}_c^{VSC}. \tag{A.1}$$

The SOCP model replaces this equality by $(I_c^{VSC})^2 \le l_c^{VSC}$, as in (2.31a). A loose inequality can make the current used in the loss model inconsistent with the squared current used in other converter constraints. Two changes are used together to reduce this gap.

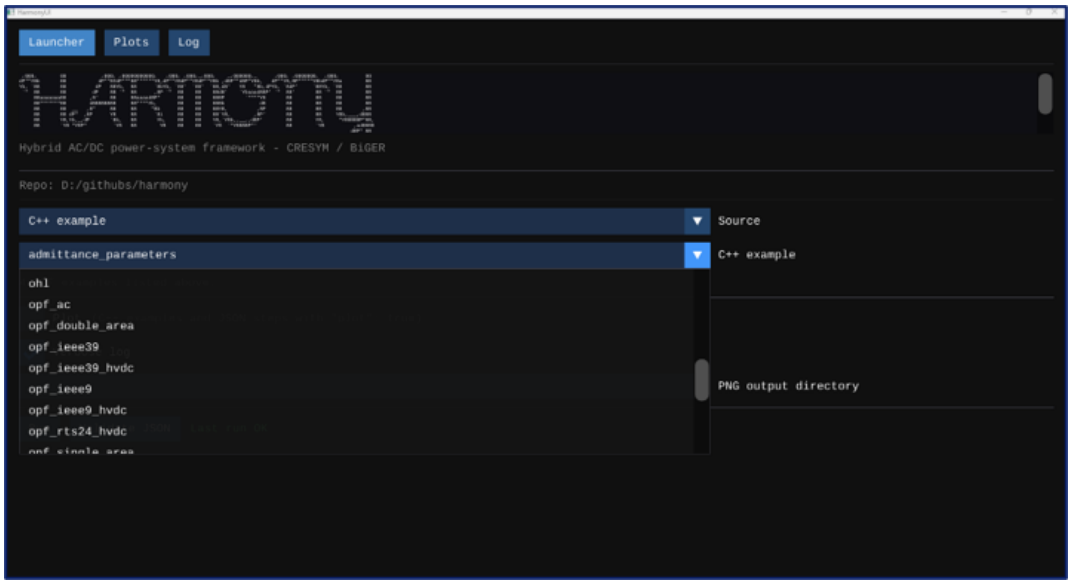


Figure A.2: *Harmony* UI



Figure A.3: OPF results presented in *Harmony* UI

1) Include converter-station losses in the objective. Let F_{gen} denote the generation-cost objective in Eq. (2.33), evaluated using the physical power bases. The objective becomes

$$\min \; F_{\text{gen}} + \lambda \sum_{c \in \mathcal{N}^{VSC}} P^{\text{loss}}_{\text{station},c}. \tag{A.2}$$

The current **Harmony** implementation includes both converter internal losses and the active losses of the AC interface:

$$\widetilde{p}^{VSC}_{\text{loss},c} = a^{VSC}_{\text{loss},c} + b^{VSC}_{\text{loss},c} I^{VSC}_c + c^{VSC}_{\text{loss},c} l^{VSC}_c, \tag{A.3}$$

$$P^{\text{loss}}_{\text{station},c} = S^{DC}_{\text{base}} \big(\widetilde{p}^{VSC}_{\text{loss},c} + p^{VSC}_s + p^{VSC}_c \big). \tag{A.4}$$

Here, the powers inside parentheses are in per unit, S^{DC}_{base} converts them to MW, and s is the PCC associated with converter c. The signs follow the OPF port conventions: $p^{VSC}_s + p^{VSC}_c$ is the AC-interface loss. The original physical internal-loss expression is $a^{VSC}_{\text{loss},c} + b^{VSC}_{\text{loss},c} I^{VSC}_c + c^{VSC}_{\text{loss},c} (I^{VSC}_c)^2$; Eq. (A.3) uses the lifted variable l^{VSC}_c. The reported comparisons use generation-cost and station-loss weights of 1 and 10, respectively, with generation cost in \$/h and station losses in MW. Thus, λ carries the corresponding cost-per-power units. This setting is a tuning choice for these examples, rather than a universal value for every network.

2) Bound the squared current using a piecewise envelope. For each converter, divide $[0, \overline{I}^{VSC}_c]$ into K intervals $[a_{c,k}, b_{c,k}]$. Introduce a binary interval selector $z_{c,k}$ and a continuous segment-current variable $I^{VSC}_{c,k}$, and impose

$$\sum_{k=1}^{K} z_{c,k} = 1, \qquad z_{c,k} \in \{0,1\}, \tag{A.5}$$

$$a_{c,k} z_{c,k} \le I^{VSC}_{c,k} \le b_{c,k} z_{c,k}, \qquad k = 1, \dots, K, \tag{A.6}$$

$$I^{VSC}_c = \sum_{k=1}^{K} I^{VSC}_{c,k}, \tag{A.7}$$

$$l^{VSC}_c \ge \sum_{k=1}^{K} (I^{VSC}_{c,k})^2, \tag{A.8}$$

$$l^{VSC}_c \le \sum_{k=1}^{K} \big[(a_{c,k} + b_{c,k}) I^{VSC}_{c,k} - a_{c,k} b_{c,k} z_{c,k} \big]. \tag{A.9}$$

Exactly one interval is active. Within that interval, the quadratic lower bound and the secant upper bound restrict the gap in Eq. (A.1). The implementation and the comparisons below use four equal intervals, with $a_{c,k} = (k-1)\overline{I}^{VSC}_c / K$ and $b_{c,k} = k \overline{I}^{VSC}_c / K$. For an active interval, the envelope implies

$$0 \le l^{VSC}_c - (I^{VSC}_c)^2 \le \frac{(b_{c,k} - a_{c,k})^2}{4}. \tag{A.10}$$

The formulation therefore remains an approximation. More intervals can tighten this bound, at the cost of additional binary variables. The remaining power-flow constraints retain their relaxed structure.

For comparison, SOCP-a denotes the original relaxation; SOCP-b adds the station-loss objective only; MISOCP-a adds the current envelope only; and MISOCP-b includes both modifications.

Test systems and relaxation residuals

Five test systems combine the MATPOWER AC networks listed in Table 27 with the same five-node, five-branch DC network, five VSCs, and two aggregated RES plants with active-power limits of 80 and 70 MW. VSC1–3 connect to the listed AC PCC buses; VSC4–5 connect the RES plants to the DC network. These integration tests use the settings specified here and are separate from the standalone comparisons in the preceding subsection.

Table 27: Overview of the AC/DC test systems.

AC network	AC buses	AC branches	AC generators	AC load [MW]	AC PCC buses (VSC1–3)
`case9`	9	9	3	315.00	4, 6, 8
`case57`	57	80	7	1250.80	11, 21, 45
`case89pegase`	89	210	12	5727.89	1367, 4423, 6833
`case39`	39	46	10	6254.23	2, 6, 10
`case118`	118	186	54	4242.00	68, 5, 17

All systems use a five-node, five-branch DC network, five VSCs, and two aggregated RES plants with active-power limits of 80 and 70 MW. VSC4–5 connect the RES plants to the DC network. The AC grids of systems A, B, C, D, E are based on MATPOWER `case9`, `case57`, `case89pegase`, `case39`, and `case118`, respectively.

To assess the relaxed equalities, define

$$r_e = \frac{|L_e(x^{\mathrm{rel}}) - R_e(x^{\mathrm{rel}})|}{|R_e(x^{\mathrm{NLP}})|}\,100\%, \tag{A.11}$$

where L_e and R_e are the two sides of the corresponding original equality. Tables 28–47 report the mean, median, and maximum residuals for all four formulations and all five systems. The percentages use the corresponding NLP quantities as denominators; for example, the current-equality residual is normalized by the NLP squared current, rather than the converter current rating. Residuals below 0.001% are reported as ≈ 0.

The combined modification in MISOCP-b substantially reduces the maximum residual of $(I_c^{VSC})^2 = l_c^{VSC}$ relative to SOCP-a: from 102.430 to 7.54530% in System A, from 120.401 to 4.59118% in B, from 109.663 to 6.28993% in C, from 107.191 to 8.68027% in D, and from 177.434 to 40.0383% in E. The VSC AC-interface voltage-product residual remains below 0.001% in all five systems. These results show the benefit of the modification in MISOCP-b.

Table 28: Relaxation Residuals for the SOCP-a model (System A)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.328101	≈ 0	2.02884
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	1.33049	0.0108448	6.49687
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	1.33096	0.0107065	6.49891
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00247436	0.00151882	0.00748245
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	100.496	100.002	102.43

≈ 0 denotes a relative equality residual below 0.001%.

Table 29: Relaxation Residuals for the SOCP-b model (System A)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.482371	≈ 0	2.76281
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.051391	0.00709332	0.125485
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0515079	0.00699864	0.12578
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00103297	0.00122788	0.00131645
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	100.747	100.001	102.357

≈ 0 denotes a relative equality residual below 0.001%.

Table 30: Relaxation Residuals for the MISOCP-a model (System A)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.465821	≈ 0	2.77554
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	1.23967	0.00381243	6.15669
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	1.24009	0.00376205	6.15876
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	7.63406	2.4655	28.8656

≈ 0 denotes a relative equality residual below 0.001%.

Table 31: Relaxation Residuals for the MISOCP-b model (System A)

Involved Equality	**Mean** [%]	**Median** [%]	**Maximum** [%]
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.401476	≈ 0	2.65516
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.16569	0.0242411	0.553605
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.166048	0.0239008	0.55498
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.0016002	≈ 0	0.00462921
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	4.19794	3.95873	7.5453

≈ 0 denotes a relative equality residual below 0.001%.

Table 32: Relaxation Residuals for the SOCP-a model (System B)

Involved Equality	**Mean** [%]	**Median** [%]	**Maximum** [%]
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0741923	≈ 0	3.02802
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.00494075	0.00347419	0.0134569
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.00495432	0.00343757	0.013534
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00135075	≈ 0	0.00437287
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	97.5139	100.001	120.401

≈ 0 denotes a relative equality residual below 0.001%.

Table 33: Relaxation Residuals for the SOCP-b model (System B)

Involved Equality	**Mean** [%]	**Median** [%]	**Maximum** [%]
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0740625	≈ 0	2.99645
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.00363625	0.00360255	0.010317
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.00364162	0.00356351	0.0103756
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	≈ 0	≈ 0	0.00180213
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	97.5511	100.000	119.585

≈ 0 denotes a relative equality residual below 0.001%.

Table 34: Relaxation Residuals for the MISOCP-a model (System B)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.073718	≈ 0	3.00902
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.0018926	≈ 0	0.00514646
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.00189917	≈ 0	0.00517476
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00252217	≈ 0	0.0118484
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	2.18938	1.1323	4.45418

≈ 0 denotes a relative equality residual below 0.001%.

Table 35: Relaxation Residuals for the MISOCP-b model (System B)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0732563	≈ 0	2.96486
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.00158039	≈ 0	0.00370441
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.00158503	≈ 0	0.00372555
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	≈ 0	≈ 0	0.0014816
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	2.60299	2.34591	4.59118

≈ 0 denotes a relative equality residual below 0.001%.

Table 36: Relaxation Residuals for the SOCP-a model (System C)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0080082	≈ 0	1.57132
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.166597	0.0832362	0.540712
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.16642	0.0841893	0.537564
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.0198212	0.01025	0.0419172
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	102.765	100.014	109.663

≈ 0 denotes a relative equality residual below 0.001%.

Table 37: Relaxation Residuals for the SOCP-b model (System C)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.00976539	≈ 0	1.93367
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.23982	0.0582808	0.93792
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.240754	0.0572258	0.942472
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00437211	0.00434382	0.00651589
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	99.5094	99.4282	100.003

≈ 0 denotes a relative equality residual below 0.001%.

Table 38: Relaxation Residuals for the MISOCP-a model (System C)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.00774794	≈ 0	1.56275
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.0934791	0.0445183	0.310025
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0933754	0.0449961	0.30833
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	8.95593	0.00670503	38.4035
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	0.801817	0.00989729	3.82776

≈ 0 denotes a relative equality residual below 0.001%.

Table 39: Relaxation Residuals for the MISOCP-b model (System C)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0101295	≈ 0	1.97456
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	1.10973	0.133836	5.04885
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	1.11436	0.131517	5.07301
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00187747	0.00172031	0.0031037
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	3.80159	3.84907	6.28993

≈ 0 denotes a relative equality residual below 0.001%.

Table 40: Relaxation Residuals for the SOCP-a model (System D)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0343084	0.00101521	1.52606
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.608285	0.216849	2.25647
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.610679	0.214047	2.26941
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.149342	0.17368	0.272668
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	101.157	100.029	107.191

≈ 0 denotes a relative equality residual below 0.001%.

Table 41: Relaxation Residuals for the SOCP-b model (System D)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.034439	0.00119586	1.51113
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.393707	0.164023	1.41205
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.394962	0.161745	1.41954
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.0965422	0.0842071	0.197437
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	100.86	100.013	104.54

≈ 0 denotes a relative equality residual below 0.001%.

Table 42: Relaxation Residuals for the MISOCP-a model (System D)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0338293	≈ 0	1.55253
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.0221545	0.0130695	0.0562598
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0222269	0.0129	0.0565721
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	11.4659	0.00134516	55.906
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	1.00086	0.0208816	3.85239

≈ 0 denotes a relative equality residual below 0.001%.

Table 43: Relaxation Residuals for the MISOCP-b model (System D)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.0297409	≈ 0	1.36382
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.021572	0.0108309	0.0679199
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0216396	0.0106809	0.068278
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00844984	0.00327186	0.0285272
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	2.7367	1.12852	8.68027

≈ 0 denotes a relative equality residual below 0.001%.

Table 44: Relaxation Residuals for the SOCP-a model (System E)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.113843	≈ 0	13.0968
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.0170889	0.00141321	0.0818427
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0170406	0.00143871	0.081566
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00411362	≈ 0	0.0178372
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	113.823	100.001	177.434

≈ 0 denotes a relative equality residual below 0.001%.

Table 45: Relaxation Residuals for the SOCP-b model (System E)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.112971	≈ 0	12.9848
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.00644894	≈ 0	0.0286559
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.00642447	≈ 0	0.0284999
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	0.00131366	≈ 0	0.00505352
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	109.899	100	157.775

≈ 0 denotes a relative equality residual below 0.001%.

Table 46: Relaxation Residuals for the MISOCP-a model (System E)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.122766	≈ 0	14.6386
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.0324716	0.00586745	0.13772
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0324249	0.0059634	0.137324
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	6.39951	0.00468055	31.9762
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	11.8478	1.12917	54.2521

≈ 0 denotes a relative equality residual below 0.001%.

Table 47: Relaxation Residuals for the MISOCP-b model (System E)

Involved Equality	**Mean [%]**	**Median [%]**	**Maximum [%]**
$\left(c_{ij}^{AC}\right)^2 + \left(s_{ij}^{AC}\right)^2 = c_{ii}^{AC} c_{jj}^{AC}$	0.121209	≈ 0	14.363
$\left(p_{jk}^{MTDC}\right)^2 = l_{jk}^{MTDC} u_j^{MTDC}$	0.0176664	0.00683134	0.0673914
$\left(p_{kj}^{MTDC}\right)^2 = l_{kj}^{MTDC} u_k^{MTDC}$	0.0176431	0.00693817	0.0670959
$\left(c_{sc}^{VSC}\right)^2 + \left(s_{sc}^{VSC}\right)^2 = c_{ss}^{VSC} c_{cc}^{VSC}$	≈ 0	≈ 0	≈ 0
$\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2 = c_{cc}^{VSC} l_c^{VSC}$	6.80001	0.00551551	33.9823
$\left(I_c^{VSC}\right)^2 = l_c^{VSC}$	9.00469	1.12856	40.0383

≈ 0 denotes a relative equality residual below 0.001%.

Generation cost and generator dispatch

Tables 48–50 report the individual generator active powers, total active generation, and generation cost in MISOCP-b. The generation-cost differences from the NLP benchmark are 0.0136, 0.2048, 0.1329, 0.0165, and 0.2460% for Systems A–E, respectively; total-generation differences are below 0.16% in all five systems.

Table 48: Generator active-power dispatch and generation-cost comparison between MISOCP-b and the NLP benchmark.

Case System	Quantity	Bus Number	NLP	MISOCP-b	Absolute Error	Difference [%]
System A	$P_{g,1}$ (MW)	1	43.9131	44.0179	0.1047	0.2385
	$P_{g,2}$ (MW)	2	77.3865	77.4069	0.0204	0.0264
	$P_{g,3}$ (MW)	3	54.7562	54.6532	0.1031	0.1882
	$\sum_g P_g$ (MW)	—	176.0558	176.0779	0.0221	0.0125
	C_{gen} (\$/h)	—	2540.6274	2540.9727	0.3453	0.0136
System B	$P_{g,1}$ (MW)	1	137.2601	136.6234	0.6366	0.4638
	$P_{g,2}$ (MW)	2	53.1967	50.0533	3.1433	5.9089
	$P_{g,3}$ (MW)	3	42.8063	42.7478	0.0585	0.1368
	$P_{g,4}$ (MW)	6	44.9015	46.0902	1.1887	2.6474
	$P_{g,5}$ (MW)	8	444.3612	445.6475	1.2863	0.2895
	$P_{g,6}$ (MW)	9	55.3428	55.7013	0.3585	0.6477
	$P_{g,7}$ (MW)	12	345.0478	344.3354	0.7123	0.2064
	$\sum_g P_g$ (MW)	—	1122.9164	1121.1990	1.7174	0.1529
	C_{gen} (\$/h)	—	35754.4656	35681.2490	73.2167	0.2048
System C	$P_{g,1}$ (MW)	913	1642.3032	1564.9681	77.3351	4.7089
	$P_{g,2}$ (MW)	2107	1153.4932	1125.1793	28.3139	2.4546
	$P_{g,3}$ (MW)	2267	166.6700	166.6753	0.0053	0.0032
	$P_{g,4}$ (MW)	3659	666.6700	666.6911	0.0211	0.0032
	$P_{g,5}$ (MW)	4586	44.1281	99.9970	55.8689	126.6062
	$P_{g,6}$ (MW)	5097	21.2300	21.2150	0.0150	0.0706
	$P_{g,7}$ (MW)	6233	529.7280	581.1119	51.3840	9.7001
	$P_{g,8}$ (MW)	6798	333.3300	333.8024	0.4724	0.1417
	$P_{g,9}$ (MW)	7279	100.0000	99.9956	0.0044	0.0044
	$P_{g,10}$ (MW)	7960	208.6602	203.7172	4.9429	2.3689
	$P_{g,11}$ (MW)	8605	600.0000	599.9925	0.0075	0.0013
	$P_{g,12}$ (MW)	9239	208.6506	203.9749	4.6757	2.2409
	$\sum_g P_g$ (MW)	—	5674.8633	5667.3205	7.5428	0.1329
	C_{gen} (\$/h)	—	5674.8633	5667.3205	7.5428	0.1329
System D	$P_{g,1}$ (MW)	30	645.9994	644.5924	1.4069	0.2178
	$P_{g,2}$ (MW)	31	646.0000	645.9989	0.0011	≈ 0
	$P_{g,3}$ (MW)	32	644.5740	644.3839	0.1901	0.0295
	$P_{g,4}$ (MW)	33	632.6745	632.8019	0.1274	0.0201
	$P_{g,5}$ (MW)	34	508.0000	507.9999	0.0001	≈ 0
	$P_{g,6}$ (MW)	35	637.6303	637.7515	0.1213	0.0190
	$P_{g,7}$ (MW)	36	580.0000	579.9998	0.0002	≈ 0
	$P_{g,8}$ (MW)	37	564.0000	563.9996	0.0004	≈ 0
	$P_{g,9}$ (MW)	38	631.7920	633.3540	1.5619	0.2472
	$P_{g,10}$ (MW)	39	664.4450	663.7859	0.6591	0.0992
	$\sum_g P_g$ (MW)	—	6155.1152	6154.6678	0.4474	0.0073
	C_{gen} (\$/h)	—	39950.1803	39943.5798	6.6006	0.0165

Table 49: Generator active-power dispatch and generation-cost comparison for System E: generators 1–27.

Case System	Quantity	Bus Number	NLP	MISOCP-b	Absolute Error	Difference [%]
System E	$P_{g,1}$ (MW)	1	12.8395	10.6361	2.2033	17.1607
	$P_{g,2}$ (MW)	4	≈ 0	0.0004	0.0004	—
	$P_{g,3}$ (MW)	6	≈ 0	0.0009	0.0009	—
	$P_{g,4}$ (MW)	8	≈ 0	0.0004	0.0004	—
	$P_{g,5}$ (MW)	10	392.1679	390.7805	1.3874	0.3538
	$P_{g,6}$ (MW)	12	83.9569	83.6186	0.3383	0.4029
	$P_{g,7}$ (MW)	15	5.4181	0.0081	5.4099	99.8498
	$P_{g,8}$ (MW)	18	≈ 0	0.0022	0.0022	—
	$P_{g,9}$ (MW)	19	9.0770	2.2630	6.8141	75.0692
	$P_{g,10}$ (MW)	24	≈ 0	0.0015	0.0015	—
	$P_{g,11}$ (MW)	25	191.2215	194.4121	3.1906	1.6685
	$P_{g,12}$ (MW)	26	275.5571	277.4790	1.9219	0.6975
	$P_{g,13}$ (MW)	27	2.3603	8.1096	5.7493	243.5805
	$P_{g,14}$ (MW)	31	7.1570	7.1604	0.0033	0.0465
	$P_{g,15}$ (MW)	32	6.2525	10.9834	4.7309	75.6641
	$P_{g,16}$ (MW)	34	≈ 0	0.0010	0.0010	—
	$P_{g,17}$ (MW)	36	2.1558	0.0014	2.1545	99.9366
	$P_{g,18}$ (MW)	40	44.8891	37.2956	7.5934	16.9160
	$P_{g,19}$ (MW)	42	38.2281	38.6724	0.4443	1.1622
	$P_{g,20}$ (MW)	46	18.9340	19.0058	0.0718	0.3792
	$P_{g,21}$ (MW)	49	192.2651	193.8959	1.6308	0.8482
	$P_{g,22}$ (MW)	54	49.3805	48.9778	0.4028	0.8157
	$P_{g,23}$ (MW)	55	29.1298	20.2207	8.9091	30.5842
	$P_{g,24}$ (MW)	56	29.3723	20.9586	8.4137	28.6450
	$P_{g,25}$ (MW)	59	148.7878	145.4017	3.3861	2.2758
	$P_{g,26}$ (MW)	61	147.3625	147.4365	0.0739	0.0502
	$P_{g,27}$ (MW)	62	≈ 0	0.0004	0.0004	—

Table 50: Generator active-power dispatch and generation-cost comparison for System E: generators 28–54.

Case System	Quantity	Bus Number	NLP	MISOCP-b	Absolute Error	Difference [%]
System E	$P_{g,28}$ (MW)	65	349.4262	350.4057	0.9795	0.2803
	$P_{g,29}$ (MW)	66	346.2865	351.3019	5.0154	1.4483
	$P_{g,30}$ (MW)	69	448.9264	461.9527	13.0263	2.9017
	$P_{g,31}$ (MW)	70	≈ 0	0.0019	0.0019	—
	$P_{g,32}$ (MW)	72	≈ 0	0.0073	0.0073	—
	$P_{g,33}$ (MW)	73	≈ 0	0.0029	0.0029	—
	$P_{g,34}$ (MW)	74	14.7950	18.9468	4.1518	28.0622
	$P_{g,35}$ (MW)	76	21.4353	23.9580	2.5227	11.7688
	$P_{g,36}$ (MW)	77	≈ 0	0.0005	0.0005	—
	$P_{g,37}$ (MW)	80	428.2228	427.9052	0.3176	0.0742
	$P_{g,38}$ (MW)	85	≈ 0	0.0002	0.0002	—
	$P_{g,39}$ (MW)	87	3.6183	3.6100	0.0084	0.2313
	$P_{g,40}$ (MW)	89	500.9434	501.1514	0.2081	0.0415
	$P_{g,41}$ (MW)	90	≈ 0	0.0003	0.0003	—
	$P_{g,42}$ (MW)	91	≈ 0	0.0003	0.0003	—
	$P_{g,43}$ (MW)	92	≈ 0	0.0002	0.0002	—
	$P_{g,44}$ (MW)	99	≈ 0	0.0004	0.0004	—
	$P_{g,45}$ (MW)	100	230.6738	230.6656	0.0082	0.0035
	$P_{g,46}$ (MW)	103	38.1811	38.1828	0.0017	0.0044
	$P_{g,47}$ (MW)	104	≈ 0	0.0033	0.0033	—
	$P_{g,48}$ (MW)	105	4.3780	3.6844	0.6936	15.8431
	$P_{g,49}$ (MW)	107	28.7071	28.3010	0.4061	1.4147
	$P_{g,50}$ (MW)	110	6.5655	5.7712	0.7943	12.0975
	$P_{g,51}$ (MW)	111	35.2177	35.1963	0.0215	0.0609
	$P_{g,52}$ (MW)	112	36.3661	35.8532	0.5129	1.4104
	$P_{g,53}$ (MW)	113	≈ 0	0.0006	0.0006	—
	$P_{g,54}$ (MW)	116	≈ 0	0.0002	0.0002	—
	$\sum_g P_g$ (MW)	—	4180.2560	4174.2280	6.0280	0.1442
	C_{gen} (\$/h)	—	124139.3356	123833.9827	305.3529	0.2460

Voltage-angle recovery and converter operating points

Tables 51 and 52 use two references. The first is the directly optimized NLP solution. The second is obtained by nonlinear power-flow reconstruction from specified MISOCP-b setpoints. For this second comparison, AC reconstruction fixes $(p^{AC}_{i,\text{gen}}, v^{AC}_i)$ at PV buses, (p^{AC}_i, q^{AC}_i) at PQ buses, and v^{AC}_i with $\theta^{AC}_i = 0$ at the slack bus. DC reconstruction fixes the reference voltage and the other nodal active injections. VSC reconstruction fixes (p^{VSC}_c, q^{VSC}_c). PCC voltages follow the reconstructed AC voltages for VSC1–3, while the optimized PCC magnitudes and zero phase are specified for VSC4–5. The remaining variables are solved from the nonlinear power flow equations.

Table 51: Voltage-magnitude and voltage-angle errors of the MISOCP-b model relative to the NLP benchmark.

Case System	Quantity	Mean	Median	Maximum
System A	AC voltage magnitude [%]	0.206315	0.00432041	1.35814
	DC voltage magnitude [%]	0.00747109	0.00836810	0.0120841
	VSC voltage magnitude [%]	0.0142826	0.00626204	0.0393606
	AC voltage angle [°]	0.216424	0.290495	0.407878
	VSC voltage angle [°]	0.140293	0.00231079	0.405901
System B	AC voltage magnitude [%]	1.77062	1.98377	2.47356
	DC voltage magnitude [%]	0.0548060	0.0542950	0.102097
	VSC voltage magnitude [%]	1.46709	1.88087	3.98058
	AC voltage angle [°]	3.72404	3.99753	9.40139
	VSC voltage angle [°]	2.48631	1.45866	6.38460
System C	AC voltage magnitude [%]	0.682873	0.705366	1.99691
	DC voltage magnitude [%]	0.00807336	0.00827650	0.0134839
	VSC voltage magnitude [%]	0.635162	0.710278	1.47083
	AC voltage angle [°]	1.51906	1.27929	5.95600
	VSC voltage angle [°]	0.766060	0.264332	2.57731
System D	AC voltage magnitude [%]	0.135982	0.0962373	0.944754
	DC voltage magnitude [%]	0.0515721	0.0527548	0.0991961
	VSC voltage magnitude [%]	0.166661	0.118616	0.812173
	AC voltage angle [°]	0.964162	0.435833	3.30525
	VSC voltage angle [°]	0.605527	0.00894011	3.00806
System E	AC voltage magnitude [%]	0.396937	0.286345	3.06185
	DC voltage magnitude [%]	0.0333955	0.0325373	0.0653385
	VSC voltage magnitude [%]	0.352398	0.0131659	1.58418
	AC voltage angle [°]	8.69227	9.30833	24.7936
	VSC voltage angle [°]	5.58828	2.35441	13.8606

The NLP benchmark minimizes generation cost only. VSC statistics include both terminals of all five converters. Relative voltage-magnitude errors below 0.001% are reported as ≈ 0.

Table 52: Voltage-magnitude and voltage-angle errors of MISOCP-b relative to nonlinear power-flow references reconstructed from its optimized solution.

Case System	Quantity	Mean	Median	Maximum
System A	AC voltage magnitude [%]	0.413671	0.422460	1.02036
	DC voltage magnitude [%]	≈ 0	≈ 0	≈ 0
	VSC voltage magnitude [%]	0.377434	0.284108	1.02036
	AC voltage angle [°]	0.153705	0.155070	0.307489
	VSC voltage angle [°]	0.0927101	0.0113371	0.307489
System B	AC voltage magnitude [%]	0.821358	0.847923	2.50095
	DC voltage magnitude [%]	≈ 0	≈ 0	≈ 0
	VSC voltage magnitude [%]	0.649651	0.481348	1.47430
	AC voltage angle [°]	3.57058	3.80890	9.26455
	VSC voltage angle [°]	2.15539	0.798686	6.28371
System C	AC voltage magnitude [%]	0.0639968	0.0297336	0.534680
	DC voltage magnitude [%]	≈ 0	≈ 0	≈ 0
	VSC voltage magnitude [%]	0.0415542	≈ 0	0.203398
	AC voltage angle [°]	1.65035	1.44212	6.30536
	VSC voltage angle [°]	0.641725	0.00540925	2.59252
System D	AC voltage magnitude [%]	0.0691538	0.0581343	0.35348
	DC voltage magnitude [%]	≈ 0	≈ 0	≈ 0
	VSC voltage magnitude [%]	0.0585358	0.0500333	0.185352
	AC voltage angle [°]	0.915389	0.376395	3.27292
	VSC voltage angle [°]	0.518542	0.00101493	2.59565
System E	AC voltage magnitude [%]	0.0423954	≈ 0	0.699671
	DC voltage magnitude [%]	≈ 0	≈ 0	≈ 0
	VSC voltage magnitude [%]	0.0590685	0.00155728	0.285384
	AC voltage angle [°]	9.06913	9.53452	25.3954
	VSC voltage angle [°]	5.7307	2.04967	14.0226

AC reconstruction fixes optimized $(p^{AC}_{i,\mathrm{gen}}, v^{AC}_i)$ at PV buses, (p^{AC}_i, q^{AC}_i) at PQ buses, and v^{AC}_i with $\theta^{AC}_i = 0°$ at the slack AC bus. DC reconstruction fixes optimized v^{MTDC}_j at the reference DC bus and p^{MTDC}_j elsewhere. VSC reconstruction fixes optimized (p^{VSC}_c, q^{VSC}_c). $(v^{VSC}_s, \theta^{VSC}_s)$ follow reconstructed $(v^{AC}_i, \theta^{AC}_i)$ for VSC#1–#3, while optimized $v^{VSC}_s = v^{res}_0$ and $\theta^{VSC}_s = 0°$ are specified for VSC#4–#5. Remaining variables are solved from the nonlinear power-flow equations of the respective subnetworks. Relative voltage-magnitude errors below 0.001% are reported as ≈ 0.

Voltage magnitudes generally agree more closely than voltage angles. In System E, the maximum AC-angle difference is 24.7936° relative to the NLP benchmark and 25.3954° relative to the reconstructed reference. **Accurate phase recovery therefore remains an important limitation, even where the voltage magnitudes and generation costs agree closely.**

For a separate check of port-power consistency, the original nonlinear VSC and DC power equations are evaluated at the voltage magnitudes optimized by MISOCP-b and the recovered angles. This calculation uses those voltages directly, without re-optimizing the dispatch or correcting the voltages through another OPF solve. Tables 53–57 provide the full comparisons for all five converters in each system. The optimized and recalculated port powers are close at the precision reported, including in Systems B and D, where the differences from the NLP operating points are larger. **These results provide preliminary support for the feasibility of using operating points obtained from the relaxed OPF formulation to initialize VSC models in Harmony**. improvements are nevertheless needed to reduce the remaining relaxation residuals and improve initialization reliability across different operating conditions and converter control configurations. **Port-power consistency alone does not establish feasibility of the entire network or system stability.**

Table 53: Converter-station power comparisons of the optimized and nonlinearly recalculated MISOCP-b powers against the NLP benchmark (System A).

VSC	Quantity	NLP	MISOCP-b optimized			MISOCP-b recalculated		
			Value	Absolute Error	Difference [%]	Value	Absolute Error	Difference [%]
VSC1	p_s^{VSC} [MW]	−62.6191	−62.6812	0.0621048	0.0991787	−62.6811	0.0620205	0.0990441
	q_s^{VSC} [Mvar]	−10.2693	−10.2457	0.0235995	0.229806	−10.2453	0.0240285	0.233983
	p_j^{MTDC} [MW]	−63.679	−63.7389	0.0598669	0.0940135	−63.7389	0.0598927	0.0940541
VSC2	p_s^{VSC} [MW]	−29.5705	−30.488	0.917509	3.10278	−30.488	0.917464	3.10263
	q_s^{VSC} [Mvar]	≈ 0	0.018154	0.0181541	—	0.018436	0.018436	—
	p_j^{MTDC} [MW]	−29.9158	−30.8422	0.926367	3.09658	−30.8422	0.926403	3.0967
VSC3	p_s^{VSC} [MW]	−50.1812	−49.2077	0.973433	1.93984	−49.2077	0.973511	1.93999
	q_s^{VSC} [Mvar]	≈ 0	0.00223406	0.00223005	—	0.00267309	0.00266908	—
	p_j^{MTDC} [MW]	−50.8643	−49.8683	0.995982	1.95812	−49.8683	0.995954	1.95806
VSC4	p_s^{VSC} [MW]	80	80.0001	≈ 0	≈ 0	80	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	0.0674648	0.0674655	—	0.0679872	0.0679878	—
	p_j^{MTDC} [MW]	78.6351	78.6365	0.00134343	0.00170843	78.6364	0.00132135	0.00168035
VSC5	p_s^{VSC} [MW]	70	70	≈ 0	≈ 0	69.9999	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	−0.00351854	0.00352176	—	−0.00298896	0.00299218	—
	p_j^{MTDC} [MW]	68.9054	68.9091	0.00372061	0.00539959	68.9091	0.00369676	0.00536499

Absolute errors and percentage differences in both column groups use the NLP benchmark. Recalculated powers are evaluated at the optimized voltage magnitudes and recovered angles. Power values with $|x| < 10^{-3}$ MW/Mvar and absolute errors below 10^{-3} MW/Mvar are reported as ≈ 0. Percentage differences below 0.001% are reported as ≈ 0.

Table 54: Converter-station power comparisons of the optimized and nonlinearly recalculated MISOCP-b powers against the NLP benchmark (System B).

VSC	Quantity	NLP	MISOCP-b optimized			MISOCP-b recalculated		
			Value	Absolute Error	Difference [%]	Value	Absolute Error	Difference [%]
VSC1	p_s^{VSC} [MW]	−51.3993	−47.9898	3.40953	6.63342	−47.9898	3.40953	6.63342
	q_s^{VSC} [Mvar]	−2.62725	−10.6871	8.05986	306.78	−10.6871	8.05986	306.78
	p_j^{MTDC} [MW]	−52.204	−48.7175	3.48644	6.6785	−48.7175	3.48644	6.6785
VSC2	p_s^{VSC} [MW]	−30.9781	−27.6086	3.36943	10.8768	−27.6086	3.36943	10.8768
	q_s^{VSC} [Mvar]	0.208996	−3.53818	3.74717	1792.94	−3.53817	3.74717	1792.94
	p_j^{MTDC} [MW]	−31.3507	−27.9314	3.41933	10.9067	−27.9314	3.41933	10.9067
VSC3	p_s^{VSC} [MW]	−60.0318	−66.8456	6.81383	11.3504	−66.8456	6.81383	11.3504
	q_s^{VSC} [Mvar]	8.19559	−9.13241	17.328	211.431	−9.1324	17.328	211.431
	p_j^{MTDC} [MW]	−60.9837	−67.938	6.9543	11.4035	−67.938	6.95431	11.4035
VSC4	p_s^{VSC} [MW]	80	80	≈ 0	≈ 0	80	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	≈ 0	≈ 0	—	≈ 0	≈ 0	—
	p_j^{MTDC} [MW]	78.6351	78.6364	0.001238	0.00157437	78.6364	0.00123705	0.00157315
VSC5	p_s^{VSC} [MW]	70	70	≈ 0	≈ 0	70	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	≈ 0	≈ 0	—	≈ 0	≈ 0	—
	p_j^{MTDC} [MW]	68.9054	68.9091	0.00371802	0.00539584	68.9091	0.0037172	0.00539464

Absolute errors and percentage differences in both column groups use the NLP benchmark. Recalculated powers are evaluated at the optimized voltage magnitudes and recovered angles. Power values with $|x| < 10^{-3}$ MW/Mvar and absolute errors below 10^{-3} MW/Mvar are reported as ≈ 0. Percentage differences below 0.001% are reported as ≈ 0.

Table 55: Converter-station power comparisons of the optimized and nonlinearly recalculated MISOCP-b powers against the NLP benchmark (System C).

VSC	Quantity	NLP	MISOCP-b optimized			MISOCP-b recalculated		
			Value	Absolute Error	Difference [%]	Value	Absolute Error	Difference [%]
VSC1	p_s^{VSC} [MW]	−46.1809	−46.6083	0.427437	0.925572	−46.6082	0.427348	0.925379
	q_s^{VSC} [Mvar]	−0.0159698	−0.049581	0.0336112	210.467	−0.0487077	0.0327379	204.998
	p_j^{MTDC} [MW]	−46.8136	−47.2357	0.42205	0.901553	−47.236	0.422323	0.902137
VSC2	p_s^{VSC} [MW]	−48.591	−49.1791	0.588075	1.21025	−49.1791	0.588063	1.21023
	q_s^{VSC} [Mvar]	0.197685	−0.685854	0.883539	446.943	−0.685408	0.883093	446.718
	p_j^{MTDC} [MW]	−49.2555	−49.8385	0.583057	1.18374	−49.8388	0.583367	1.18437
VSC3	p_s^{VSC} [MW]	−47.5266	−46.5411	0.985505	2.07359	−46.541	0.985582	2.07375
	q_s^{VSC} [Mvar]	−3.21182	−3.24842	0.0366036	1.13965	−3.24772	0.0358975	1.11767
	p_j^{MTDC} [MW]	−48.2504	−47.2329	1.01743	2.10865	−47.2332	1.01711	2.10798
VSC4	p_s^{VSC} [MW]	80	79.9996	≈ 0	≈ 0	79.9995	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	≈ 0	≈ 0	—	≈ 0	≈ 0	—
	p_j^{MTDC} [MW]	78.6351	78.6359	≈ 0	≈ 0	78.6352	≈ 0	≈ 0
VSC5	p_s^{VSC} [MW]	70	69.9997	≈ 0	≈ 0	69.9997	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	−0.00839733	0.00839047	—	−0.00831667	0.0083098	—
	p_j^{MTDC} [MW]	68.9054	68.9088	0.00341963	0.0049628	68.9081	0.00267958	0.00388878

Absolute errors and percentage differences in both column groups use the NLP benchmark. Recalculated powers are evaluated at the optimized voltage magnitudes and recovered angles. Power values with $|x| < 10^{-3}$ MW/Mvar and absolute errors below 10^{-3} MW/Mvar are reported as ≈ 0. Percentage differences below 0.001% are reported as ≈ 0.

Table 56: Converter-station power comparisons of the optimized and nonlinearly recalculated MISOCP-b powers against the NLP benchmark (System D).

VSC	Quantity	NLP	MISOCP-b optimized			MISOCP-b recalculated		
			Value	Absolute Error	Difference [%]	Value	Absolute Error	Difference [%]
VSC1	p_s^{VSC} [MW]	−45.9487	−52.7492	6.80054	14.8003	−52.7492	6.8005	14.8002
	q_s^{VSC} [Mvar]	5.14534	0.0071168	5.13822	99.8617	0.00730424	5.13803	99.858
	p_j^{MTDC} [MW]	−46.5678	−53.4992	6.93139	14.8845	−53.4992	6.9314	14.8846
VSC2	p_s^{VSC} [MW]	−40.4038	−37.7888	2.61503	6.47222	−37.7888	2.61505	6.47229
	q_s^{VSC} [Mvar]	−9.96311	−6.40167	3.56145	35.7463	−6.40151	3.5616	35.7479
	p_j^{MTDC} [MW]	−40.9591	−38.2782	2.6809	6.54529	−38.2783	2.68088	6.54526
VSC3	p_s^{VSC} [MW]	−56.0738	−51.917	4.15673	7.41297	−51.917	4.15675	7.41301
	q_s^{VSC} [Mvar]	−6.5686	−4.88993	1.67867	25.556	−4.8898	1.6788	25.5579
	p_j^{MTDC} [MW]	−56.9228	−52.6678	4.25502	7.47507	−52.6678	4.25501	7.47505
VSC4	p_s^{VSC} [MW]	80	80	≈ 0	≈ 0	80	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	0.00365005	0.00364849	—	0.00391364	0.00391208	—
	p_j^{MTDC} [MW]	78.6351	78.6364	0.00123862	0.00157515	78.6363	0.00122388	0.00155641
VSC5	p_s^{VSC} [MW]	70	70	≈ 0	≈ 0	70	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	0.00396888	0.00395984	—	0.00421182	0.00420278	—
	p_j^{MTDC} [MW]	68.9054	68.9091	0.00372443	0.00540514	68.9091	0.00371136	0.00538617

Absolute errors and percentage differences in both column groups use the NLP benchmark. Recalculated powers are evaluated at the optimized voltage magnitudes and recovered angles. Power values with $|x| < 10^{-3}$ MW/Mvar and absolute errors below 10^{-3} MW/Mvar are reported as ≈ 0. Percentage differences below 0.001% are reported as ≈ 0.

Table 57: Converter-station power comparisons of the optimized and nonlinearly recalculated MISOCP-b powers against the NLP benchmark (System E).

VSC	Quantity	NLP	MISOCP-b optimized			MISOCP-b recalculated		
			Value	Absolute Error	Difference [%]	Value	Absolute Error	Difference [%]
VSC1	p_s^{VSC} [MW]	−16.5261	−15.9869	0.539214	3.26281	−15.9869	0.53922	3.26284
	q_s^{VSC} [Mvar]	−0.833101	−0.929149	0.0960476	11.5289	−0.929109	0.0960081	11.5242
	p_j^{MTDC} [MW]	−16.7373	−16.1733	0.564004	3.36974	−16.1733	0.56399	3.36966
VSC2	p_s^{VSC} [MW]	−41.5821	−46.5324	4.95036	11.905	−46.5324	4.95036	11.905
	q_s^{VSC} [Mvar]	−4.05146	−4.67087	0.619414	15.2887	−4.67084	0.61938	15.2878
	p_j^{MTDC} [MW]	−42.1163	−47.2005	5.08417	12.0717	−47.2005	5.08418	12.0718
VSC3	p_s^{VSC} [MW]	−83.8801	−79.4786	4.40154	5.24742	−79.4786	4.40155	5.24743
	q_s^{VSC} [Mvar]	4.22451	1.75989	2.46462	58.341	1.75995	2.46457	58.3397
	p_j^{MTDC} [MW]	−85.4583	−80.9161	4.54221	5.31512	−80.9161	4.5422	5.31511
VSC4	p_s^{VSC} [MW]	80	80	≈ 0	≈ 0	80	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	−0.00290253	0.00290254	—	−0.00283753	0.00283753	—
	p_j^{MTDC} [MW]	78.6351	78.6364	0.00125544	0.00159654	78.6364	0.00124422	0.00158226
VSC5	p_s^{VSC} [MW]	70	70	≈ 0	≈ 0	70	≈ 0	≈ 0
	q_s^{VSC} [Mvar]	≈ 0	0.00541472	0.00541471	—	0.0054966	0.00549659	—
	p_j^{MTDC} [MW]	68.9054	68.9091	0.00374134	0.00542968	68.9091	0.00373081	0.00541439

Absolute errors and percentage differences in both column groups use the NLP benchmark. Recalculated powers are evaluated at the optimized voltage magnitudes and recovered angles. Power values with $|x| < 10^{-3}$ MW/Mvar and absolute errors below 10^{-3} MW/Mvar are reported as ≈ 0. Percentage differences below 0.001% are reported as ≈ 0.

Operating limits after nonlinear reconstruction

The operating-limit assessment fixes the non-slack generator active powers and all five VSC PCC active and reactive powers at their MISOCP-b values, then solves the coupled original AC, DC, and VSC equations. Voltages, angles, generator reactive powers, and slack active power are allowed to adjust. Tables 58–62 report the resulting voltage, generator, converter-current, and branch-capacity checks. Counts refer to violating objects divided by the total number of objects. The mean, median, and maximum are calculated over the objects that violate the applicable bounds; dashes indicate that no violation magnitudes are reported.

No violations were reported for generator active or reactive powers, VSC PCC or converter-terminal voltage magnitudes, VSC currents, or DC-branch active powers in these reconstructions. AC and DC voltage violations occur only in Systems B and C and remain below 0.008 p.u. System C has one AC-branch violation of 7.80795 MVA, equal to approximately 2.45% of that branch's 319 MVA rating. Systems B and E have no finite AC-branch capacity limits in these data, so their AC-branch compliance cannot be assessed from this comparison. **Overall, the reconstructed points show good operating-limit compliance.**

Table 58: Operating-limit violations after nonlinear reconstruction from the MISOCP-b solution (System A).

Quantity	Total	Violating /Total	Mean	Median	Maximum
AC voltage magnitude [p.u.]	9	0/9	–	–	–
DC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC PCC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC converter voltage magnitude [p.u.]	5	0/5	–	–	–
Generator active power [MW]	3	0/3	–	–	–
Generator reactive power [Mvar]	3	0/3	–	–	–
VSC current [p.u.]	5	0/5	–	–	–
AC branch apparent power [MVA]	9	0/9	–	–	–
DC branch active power [MW]	5	0/5	–	–	–

– indicates that no violation magnitudes are involved. N/A indicates that no finite capacity limits are specified.

Table 59: Operating-limit violations after nonlinear reconstruction from the MISOCP-b solution (System B).

Quantity	Total	Violating /Total	Mean	Median	Maximum
AC voltage magnitude [p.u.]	57	1/57	0.00473998	0.00473998	0.00473998
DC voltage magnitude [p.u.]	5	2/5	0.00254966	0.00254966	0.00442969
VSC PCC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC converter voltage magnitude [p.u.]	5	0/5	–	–	–
Generator active power [MW]	7	0/7	–	–	–
Generator reactive power [Mvar]	7	0/7	–	–	–
VSC current [p.u.]	5	0/5	–	–	–
AC branch apparent power [MVA]	80	N/A	–	–	–
DC branch active power [MW]	5	0/5	–	–	–

– indicates that no violation magnitudes are involved. N/A indicates that no finite capacity limits are specified.

Table 60: Operating-limit violations after nonlinear reconstruction from the MISOCP-b solution (System C).

Quantity	Total	Violating /Total	Mean	Median	Maximum
AC voltage magnitude [p.u.]	89	1/89	0.0077356	0.0077356	0.0077356
DC voltage magnitude [p.u.]	5	2/5	0.00128815	0.00128815	0.0025539
VSC PCC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC converter voltage magnitude [p.u.]	5	0/5	–	–	–
Generator active power [MW]	12	0/12	–	–	–
Generator reactive power [Mvar]	12	0/12	–	–	–
VSC current [p.u.]	5	0/5	–	–	–
AC branch apparent power [MVA]	210	1/210	7.80795	7.80795	7.80795
DC branch active power [MW]	5	0/5	–	–	–

– indicates that no violation magnitudes are involved. N/A indicates that no finite capacity limits are specified.

Table 61: Operating-limit violations after nonlinear reconstruction from the MISOCP-b solution (System D).

Quantity	Total	Violating /Total	Mean	Median	Maximum
AC voltage magnitude [p.u.]	39	0/39	–	–	–
DC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC PCC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC converter voltage magnitude [p.u.]	5	0/5	–	–	–
Generator active power [MW]	10	0/10	–	–	–
Generator reactive power [Mvar]	10	0/10	–	–	–
VSC current [p.u.]	5	0/5	–	–	–
AC branch apparent power [MVA]	46	0/46	–	–	–
DC branch active power [MW]	5	0/5	–	–	–

– indicates that no violation magnitudes are involved. N/A indicates that no finite capacity limits are specified.

Table 62: Operating-limit violations after nonlinear reconstruction from the MISOCP-b solution (System E).

Quantity	Total	Violating /Total	Mean	Median	Maximum
AC voltage magnitude [p.u.]	118	0/118	–	–	–
DC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC PCC voltage magnitude [p.u.]	5	0/5	–	–	–
VSC converter voltage magnitude [p.u.]	5	0/5	–	–	–
Generator active power [MW]	54	0/54	–	–	–
Generator reactive power [Mvar]	54	0/54	–	–	–
VSC current [p.u.]	5	0/5	–	–	–
AC branch apparent power [MVA]	186	N/A	–	–	–
DC branch active power [MW]	5	0/5	–	–	–

– indicates that no violation magnitudes are involved. N/A indicates that no finite capacity limits are specified.

Further improvements

The MISOCP-b formulation integrated into **Harmony** is an enhanced version of the AC/DC OPF model developed in **ACDC-OpFlow**. By including converter-station losses in the objective and tightening the current-square relaxation, it improves the consistency of the resulting converter operating points and is better suited to the initialization required for subsequent stability analysis in **Harmony**. Nevertheless, substantial scope for improvement remains: a successful MISOCP-b solve does not guarantee nonlinear feasibility, reliable converter initialization, or system stability. Further improvements require theoretical research into tighter converter relaxations, angle recovery with cycle-consistency guarantees, nonlinear feasibility restoration, and tractable formulations incorporating stability constraints. We will continue this research and progressively incorporate the resulting advances into future versions of **ACDC-OpFlow** and **Harmony**.

References

[1] J. Beerten, S. Cole, and R. Belmans, "Generalized steady-state vsc mtdc model for sequential ac/dc power flow algorithms," *IEEE Transactions on Power Systems*, vol. 27, no. 2, pp. 821–829, 2012.

[2] H. Ergun, J. Dave, D. Van Hertem, and F. Geth, "Optimal power flow for ac–dc grids: Formulation, convex relaxation, linear approximation, and implementation," *IEEE Transactions on Power Systems*, vol. 34, no. 4, pp. 2980–2990, 2019.

[3] D. Van Hertem, O. Gomis-Bellmunt, and J. Liang, *HVDC grids: for offshore and supergrid of the future*. John Wiley & Sons, 2016.

[4] B. Kocuk, S. S. Dey, and X. A. Sun, "Strong socp relaxations for the optimal power flow problem," *Operations Research*, vol. 64, no. 6, pp. 1177–1196, 2016.

[5] L. Gan and S. H. Low, "Optimal power flow in direct current networks," in *52nd IEEE Conference on Decision and Control*, 2013, pp. 5614–5619.

[6] M. Farivar and S. H. Low, "Branch flow model: Relaxations and convexification—part i," *IEEE Transactions on Power Systems*, vol. 28, no. 3, pp. 2554–2564, 2013.

[7] L. Gan, N. Li, U. Topcu, and S. H. Low, "Exact convex relaxation of optimal power flow in radial networks," *IEEE transactions on automatic control*, vol. 60, no. 1, pp. 72–87, 2014.